

Manual Técnico — CONDOSYS

Documento	Manual Técnico
Sistema	CONDOSYS — administración residencial
Stack	Django 6.1 + Django REST Framework + Channels
Versión	1.0
Fecha	Octubre 2026
Repositorio	https://github.com/JMJ0331/condosys
Rama de trabajo	desarrollo
Licencia	Proyecto académico / uso interno

Documento generado a partir del código fuente del proyecto. Resumen de arquitectura, modelos de datos, API REST, permisos, frontend y procedimientos de operación.

Índice

- 1. Introducción
 - 1.1 Propósito del documento
 - 1.2 Qué es CONDOSYS
 - 1.3 Alcance funcional
 - 1.4 Perfiles de usuario
 - 1.5 Convenciones del documento
- 2. Stack tecnológico y dependencias
 - 2.1 Lenguaje y runtime
 - 2.2 Dependencias del proyecto
 - 2.3 Componentes principales
 - 2.4 Dependencias declaradas pero no activas
- 3. Arquitectura del proyecto
 - 3.1 Visión general
 - 3.2 Estructura de directorios
 - 3.3 El patrón por aplicación
 - 3.4 Capa de presentación: dos superficies en la misma app
 - 3.5 Patrones de implementación recurrentes

4. Instalación, ejecución y operación

- 4.1 Requisitos previos
- 4.2 Instalación desde cero
- 4.3 Comandos de uso frecuente
- 4.4 Flujo de trabajo con git
- 4.5 Rutas principales del sistema

5. Usuarios, roles y permisos

- 5.1 Modelo de usuario
- 5.2 Estados de la cuenta
- 5.3 Conjuntos de roles
- 5.4 Mecanismo de control de acceso
- 5.5 Matriz de permisos por rol
- 5.6 Matriz de acceso por módulo
- 5.7 Sesión y autenticación

6. Modelo de datos

- 6.1 Convenciones del modelo de datos
- 6.2 Mapa relacional
- 6.3 Estructura física: Jardines, edificios y apartamentos
- 6.4 Personas: propietarios y residentes
- 6.5 Dinero: cargos de mantenimiento y pagos
- 6.6 Incidencias y su ciclo de vida
- 6.7 Ficha del residencial
- 6.8 Inventario de modelos por aplicación
- 6.9 Decisiones de diseño del modelo
- 6.10 Claves foráneas polimórficas manuales

7. API REST

- 7.1 Configuración global de DRF
- 7.2 Patrón de los ViewSets
- 7.3 Inventario de endpoints
- 7.4 Operaciones CRUD generadas por el router
- 7.5 Acciones personalizadas
- 7.6 Filtros, búsqueda y ordenamiento
- 7.7 Visibilidad en la API
- 7.8 Formato de las respuestas

8. Frontend: plantillas, CSS y JavaScript

- 8.1 Arquitectura del frontend
- 8.2 Plantillas
- 8.3 Sistema de diseño: variables.css
- 8.4 Organización de los estilos
- 8.5 Convenciones de nombres en CSS

8.6 Reglas obligatorias del frontend

8.7 JavaScript

8.8 Accesibilidad y experiencia de usuario

9. Servidor ASGI, tiempo real y tareas asíncronas

9.1 Modelo de despliegue previsto

9.2 Estado actual de ASGI

9.3 Qué falta para activar el tiempo real

9.4 Tareas asíncronas (Celery)

10. Despliegue en producción

10.1 Requisitos de producción

10.2 Checklist de despliegue

10.3 Conmutar a PostgreSQL

10.4 Seguridad en producción

10.5 Lista de verificación de seguridad antes de publicar

11. Problemas conocidos y deuda técnica

11.1 Configuración (settings.py)

11.2 URLs

11.3 Tiempo real

11.4 Modelo de datos

11.5 Seguridad y permisos

11.6 Calidad y pruebas

11.7 Orden de trabajo sugerido

12. Guía de mantenimiento y extensión

12.1 Cómo agregar un módulo nuevo

12.2 Cómo agregar un campo a un modelo

12.3 Buenas prácticas al escribir código en este proyecto

12.4 Cómo diagnosticar problemas

12.5 Comandos de verificación antes de publicar

12.6 Resumen de la arquitectura en una línea

13. Módulos y funciones incorporados en 2026

13.1 La aplicación configuracion

13.2 Modelo configuracion.IntegracionIA

13.3 Modelo login.PasswordResetToken

13.4 Recuperación de contraseña por correo

13.5 Lectura de cédula con inteligencia artificial

13.6 Variables de entorno nuevas

13.7 Impacto en los inventarios

1. Introducción

1.1 Propósito del documento

Este manual describe el sistema CONDOSYS desde el punto de vista técnico: qué hace, cómo está construido, qué decisiones técnicas lo sostienen y cómo instalarlo, operarlo y extenderlo. Todo el contenido se derivó directamente del código fuente del repositorio, de modo que el documento y el sistema evolucionan juntos.

El manual está dirigido a las personas que desarrollarán, darán soporte o administrarán el sistema: perfil de desarrollo backend, perfil de desarrollo frontend y perfil de operador.

1.2 Qué es CONDOSYS

CONDOSYS es un sistema web de administración y gestión para residenciales verticales u horizontales. Centraliza la información de apartamentos, residentes, propietarios, cuotas, pagos, incidencias y control de accesos, reemplazando los cuadernos manuales y las hojas de cálculo dispersas que normalmente se usan en la operación de una administración.

El sistema responde a un problema concreto: la administración de un residencial maneja datos sensibles (personas, documentos, pagos/saldo) y tareas recurrentes con plazos (cuotas, mantenimiento, permisos). Sin un sistema único, cada tarea se resuelve por canales distintos y se pierde trazabilidad.

1.3 Alcance funcional

El alcance funcional se organiza en los siguientes módulos, cada uno desplegado como una aplicación Django independiente pero integrada en un único proyecto:

Módulo	Función principal	Usuarios principales
Inicio (dashboard)	Resumen de estado: departamentos, incidencias abiertas, pagos por vencer, actividad reciente	Todos
Residencial	Ficha única del residencial: nombre, RNC, dirección, provincia/municipio y distribución	Administrador
Departamentos	Jerarquía Jardín → Edificio → Apartamento, con estado y propietario	Todos (lectura), Gestión (escritura)
Propietarios	Registro de propietarios y asociación con apartamentos	Gestión
Residentes	Ocupantes por apartamento: relación, fecha de ingreso, mascotas	Gestión, Propietario (lectura)
Pagos	Registro de cuotas y pagos, estados de mora, descarga de comprobante en PDF	Todos (lectura), Gestión (escritura)
Mantenimientos	Catálogo de cargos y cuotas recurrentes por periodicidad	Todos (lectura), Gestión (escritura)
Incidencias	Reporte, clasificación, asignación, seguimiento e historial de cambios de estado	Todos (reportar), Gestión (gestionar)
Solicitudes	Certificados, permisos, instalaciones y trámites documentales	Gestión
Visitantes	Registro, autorización y control de entrada/salida	Seguridad, Gestión
Áreas comunes	Catálogo de áreas comunes con tipo, capacidad, horario y estado	Todos (lectura), Gestión (escritura)
Reservas	Solicitud y aprobación de reservas de áreas comunes	Todos
Comunicados	Avisos, eventos y mantenimiento de comunicados del residencial	Todos (lectura), Gestión (escritura)
Notificaciones	Avisos dirigidos a usuarios concretos	Gestión
Reportes	Reportes administrativos, ocupación, pagos y bitácora de auditoría (CSV)	Gestión
Chat	Mensajería persistida entre usuarios y grupos	Todos
Cuentas	Alta, edición, activación y roles de usuarios; perfil propio	Administrador

Tabla 1.1 - Módulos funcionales de CONDOSYS y su acceso por rol.

1.4 Perfiles de usuario

El sistema define cinco roles, almacenados en el campo role del modelo de usuario. Cada rol habilita un conjunto de módulos y un conjunto de acciones (crear, editar, eliminar):

- Administrador (admin): acceso total. Es el único rol que autoriza el registro de nuevos usuarios y gestiona roles.
- Encargado de Administración (manager): opera la operación diaria (pagos, solicitudes, residentes, reportes) pero no administra usuarios ni estructura del sistema.
- Residente (resident): consulta sus datos, reporta incidencias y reserva áreas comunes.
- Propietario (propietario): consulta información de los apartamentos que posee.
- Seguridad / Portería (security): opera el control de visitantes y accesos; sin acceso financiero ni a datos de residentes.

NOTA — El detalle normativo de permisos por rol está en el capítulo 5. La matriz completa (qué puede y qué no puede hacer cada rol) se documenta en AGENTS.md y se resume aquí en el capítulo 5.

1.5 Convenciones del documento

- Los nombres de archivo, clases y funciones aparecen en tipo monoespaciado.
- Los bloques de código son extractos literales del repositorio.
- Las cajas ATENCIÓN señalan riesgo de operación; las cajas PENDIENTE / BUG señalan defectos conocidos del código actual.
- Las rutas se expresan siempre relativas a la raíz del proyecto Django.

2. Stack tecnológico y dependencias

2.1 Lenguaje y runtime

Elemento	Valor	Comentario
Lenguaje	Python 3.13+	Entorno de desarrollo verificado con 3.13.6
Entorno virtual	venv/ (Windows: venv\Scripts\python.exe)	Nunca se instalan paquetes en el Python del sistema
Gestión de dependencias	pip + requirements.txt	El archivo no fija versiones a propósito

ATENCIÓN — requirements.txt no fija versiones. El archivo de dependencias usa comparadores `>=` sin versiones exactas. `pip install -r requirements.txt` instala siempre la última versión compatible. Consecuencia: una instalación nueva puede diferir de la que se probó. Para reproducibilidad conviene congelar versiones con `pip freeze` en el momento de una publicación. Actualización controlada: `pip install --upgrade --dry-run -r requirements.txt` para ver qué cambios sin aplicar nada.

2.2 Dependencias del proyecto

Grupo	Paquete	Versión min.	Uso en CONDOSYS
Core	django	6.1.1	Framework base: ORM, vistas, plantillas, admin, auth
API	django-rest-framework	3.18.1	ViewSets, serializers, permisos y paginación de la API
API	django-filter	26.1	DjangoFilterBackend para filtrar y ordenar listados
Seguridad web	django-cors-headers	4.9.0	Middleware CORS para consumos desde otros orígenes
Datos	psycopg	3.3.6	Driver PostgreSQL para el despliegue de producción
Real-time	channels	4.3.2	Infraestructura ASGI para WebSockets
Real-time	channels-redis	4.3.0	Channel layer distribuido sobre Redis
Real-time	daphne	4.2.3	Servidor ASGI; es el que ejecuta runserver
Utilerías	python-dotenv	1.2.3	Carga del archivo .env en settings
Utilerías	celery	5.6.3	Cola de tareas asíncronas (configurado, sin uso activo)
Utilerías	redis	8.1.0	Cliente Redis
Utilerías	Pillow	12.3.0	Procesamiento de imágenes (avatars, fotos)
Documentos	reportlab	5.0.1	Generación del comprobante de pago en PDF
Producción	gunicorn	26.2.0	Servidor WSGI para producción
Producción	whitenoise	6.12.0	Servido de archivos estáticos sin servidor extra
Desarrollo	django-extensions	4.1	Comandos de gestión adicionales
Desarrollo	django-debug-toolbar	8.0.0	Panel de depuración (no activado)

Tabla 2.1 - Dependencias declaradas en requirements.txt.

2.3 Componentes principales

Componente	Tecnología	Responsabilidad en el sistema
Backend web	Django 6.1	MVT: URLs, vistas, plantillas, ORM y administración
API REST	Django REST Framework	ViewSets por módulo con permisos, filtros y paginación
Base de datos	SQLite (dev) / PostgreSQL (prod)	Persistencia relacional de 25 modelos
Plantillas	Django Template Language	HTML renderizado en el servidor
Estilos	CSS con variables + media queries	Sistema de diseño compartido entre módulos
Interactividad	JavaScript nativo (sin framework)	Formularios, diálogos, filtros y acciones asíncronas
PDF	ReportLab	Comprobante de pago descargable
Tiempo real	Daphne + Channels + Redis	WebSockets de chat y notificaciones (pendiente de cablear)
Tareas asíncronas	Celery + Redis	Configurado en settings, sin app Celery implementada
Despliegue	Gunicorn + WhiteNoise	Servidor de producción previsto

Tabla 2.2 - Componentes tecnológicos y su papel en la arquitectura.

2.4 Dependencias declaradas pero no activas

Los siguientes paquetes aparecen en requirements.txt y/o en settings.py pero no están integrados en INSTALLED_APPS ni en el flujo de ejecución. No son errores, pero conviene saberlo antes de planificar trabajo sobre ellos:

Paquete / config	Estado real	Qué faltaría
channels	No está en INSTALLED_APPS	Registrar la app y cablear el ProtocolTypeRouter en condosys/asgi.py
chat/routing.py + consumers.py	Existe pero es código muerto	Registrar las rutas WebSocket en asgi.py con AuthMiddlewareStack
CHANNEL_LAYERS	Configurado con RedisChannelLayer	Solo se usa si se cablea el routing ASGI
celery	Configurado (broker, serializers, timezone)	Crear celery.py, el initializer de la app y registrar tareas
django-extensions	No está en INSTALLED_APPS	Añadir la app para habilitar sus comandos
django-debug-toolbar	No está en INSTALLED_APPS	Añadir app + middleware solo en desarrollo
corsheaders	Middleware activo pero la app no está instalada	Añadir 'corsheaders' a INSTALLED_APPS si se consume desde otro origen
gunicorn + whitenoise	Solo dependencias	Crear el servicio de producción y servir los estáticos con WhiteNoise
psycopg	Driver instalado	Descomentar el bloque PostgreSQL de settings.py

Tabla 2.3 - Capacidades preparadas pero no habilitadas.

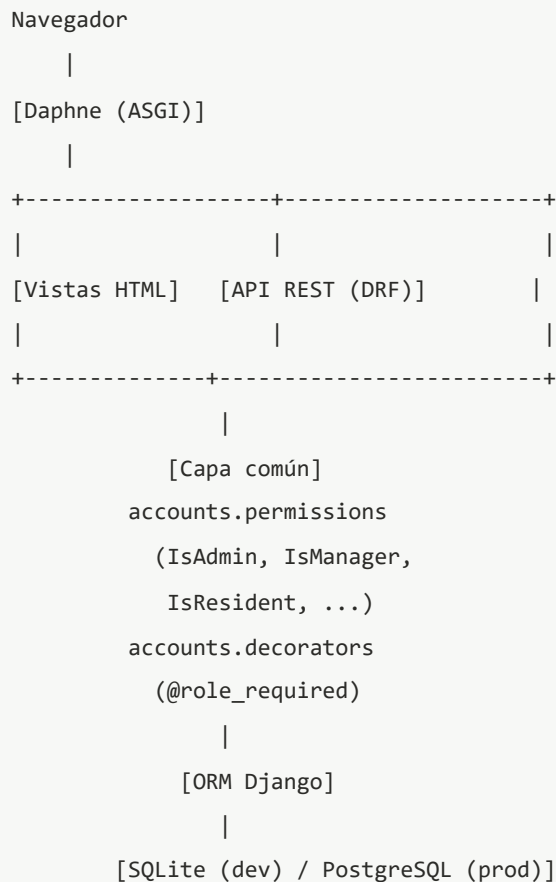
ATENCIÓN — El frontend no usa ningún framework de JavaScript. Todo el comportamiento dinámico está escrito en JavaScript nativo repartido en 29 archivos .js. No hay build step, ni bundler, ni node_modules, ni transpilador. Ventaja: no hay cadena de herramientas que mantener. Riesgo: el JavaScript del proyecto crece de forma lineal con los módulos y no tiene pruebas ni linting automático.

3. Arquitectura del proyecto

3.1 Visión general

CONDOSYS sigue el patrón clásico de Django: un único proyecto de settings y URLs que agrega múltiples aplicaciones. Cada aplicación es autónoma en cuanto a modelos, vistas, serializers, plantillas y estáticos, pero comparte el proyecto, la base de datos, el sistema de autenticación y los permisos definidos en la aplicación accounts.

Flujo de una petición en CONDOSYS



Contexto transversal: Jinja-free Django Templates, static/css (variables.css como fuente única de tokens), static/js (módulos sin framework).

3.2 Estructura de directorios

El repositorio tiene dos niveles: el proyecto Django y, dentro de él, las aplicaciones. La raíz del proyecto es el directorio que contiene manage.py.

Estructura del repositorio

```

condosys/
|-- manage.py          # utilidad de línea de comandos de Django
|-- requirements.txt    # dependencias del proyecto
|-- .gitignore         # protege .env, db.sqlite3, .env, __pycache__
|-- venv/              # entorno virtual (no versionado)
|-- db.sqlite3         # base de datos de desarrollo (no versionada)
|
|-- condosys/          # CONFIGURACIÓN del proyecto
|   |-- settings.py    # settings únicos (contiene duplicados, ver cap. 11)
|   |-- urls.py        # ROOT_URLCONF: incluye los urls de cada app
|   |-- asgi.py        # punto de entrada ASGI (sin ProtocolTypeRouter)
|   |-- wsgi.py        # punto de entrada WSGI
|   `-- accounts/      # sub-paquete auxiliar interno
|
|-- templates/         # plantillas GLOBALES
|   |-- base.html      # esqueleto común a toda la interfaz
|   `-- componentes/   # header, aside, modal, paginación, botones, ...
|
|-- static/            # estáticos COMPARTIDOS del proyecto
|   |-- css/
|       |-- variables.css # FUENTE ÚNICA de tokens de diseño
|       |-- style.css     # estilos base (importa variables.css)
|       |-- formularios.css # formularios compartidos
|       |-- tablas.css    # tablas de datos
|       |-- paginacion.css # controles de paginación
|       |-- dialogos.css  # modales y diálogos
|       |-- acciones.css  # paneles de acciones del dashboard
|       |-- entradas.css   # inputs
|       `-- mediaquerys.css # breakpoints comunes
|   |-- js/
|       |-- dialogos.js   # gestión de <dialog>
|       |-- pagos.js      # lógica compartida de pagos
|       `-- mascararas.js, contadores.js, menu-movil.js, ...
|   `-- images/icons/    # iconografía
|
`-- <app>/              # 18 aplicaciones Django
    |-- models.py        # modelos de datos
    |-- views.py         # vistas HTML + ViewSets DRF
    |-- serializers.py    # serializers de la API
    |-- forms.py         # formularios de las vistas HTML
    |-- urls.py          # rutas + DefaultRouter
    |-- admin.py         # registro en el admin de Django
    |-- tests.py         # plantilla vacía (no hay suite real)

```

```

|-- migrations/      # migraciones de esquema
|-- templates/<app>/# plantillas de la app
`-- static/css, static/js # estilos y scripts específicos del módulo

```

3.3 El patrón por aplicación

Todas las apps de dominio siguen el mismo esqueleto. Cuando agregue un módulo nuevo, copie este patrón:

Archivo	Contenido esperado	Existe en
models.py	Modelos de dominio con claves UUID y related_name explícitos	Todas salvo login e inicio
views.py	app_index() con @role_required + ViewSets DRF	Todas
serializers.py	Serializers de lectura (y de detalle cuando aplica)	Todas salvo inicio
urls.py	Rutas HTML + DefaultRouter registrado en su prefijo	Todas
forms.py	Formularios ModelForm para las vistas HTML	Mayoría
admin.py	Registro de modelos en el admin de Django	Todas
permissions.py	Permisos DRF específicos	Solo accounts (ver cap. 5)
migrations/	Migraciones de esquema	Todas salvo inicio y login
templates/	index.html como página principal del módulo	Todas
static/	CSS y JS propios del módulo	Todas salvo inicio (parcial)
tests.py	Plantilla vacía generada por startapp	Todas

Tabla 3.1 - Archivos esperados en cada aplicación de dominio.

NOTA — Convención de nombres del frontend. La página principal de cada módulo se llama siempre index.html; los demás archivos se nombran por su contenido (agregar.html, nuevo.html, perfil.html). Los estilos y scripts compartidos viven en static/css y static/js del proyecto; lo específico de un módulo queda en el static de su propia app.

3.4 Capa de presentación: dos superficies en la misma app

Cada módulo expone dos superficies complementarias:

- **Superficie HTML** - vista de función app_index que renderiza la plantilla index.html con paginación server-side, filtros por query string y formularios. Es la experiencia principal del

usuario.

- **Superficie API** - ViewSet de DRF registrado en el router que expone listado, detalle, creación, actualización y borrado en JSON, con filtros, búsqueda, ordenamiento y paginación por ?page=. La vista HTML aplica el permiso con el decorador @role_required(...); la API lo aplica con permission_classes. Los listados HTML tienen paginación de 15 elementos por página; la API usa PageNumberPagination con 20.

3.5 Patrones de implementación recurrentes

Visibilidad por usuario en los listados

Los listados HTML aplican un filtro de visibilidad para que un residente solo vea lo que le corresponde. El patrón es una función helper __visibles(user) en views.py, reutilizada por la vista y por el get_queryset() del ViewSet:

```
# Patron real (payments/views.py, estructura logica)
def _pagos_visibles(user):
    qs = Payment.objects.all()
    if user.role in ('admin', 'manager'):
        return qs

    # El residente ve los pagos de sus apartamentos y sus propios registros
    return qs.filter(Q(resident__user=user) | Q(apartment__residents__user=user)) \
        .distinct()

# El ViewSet lo reutiliza para no duplicar la regla
def get_queryset(self):
    qs = _pagos_visibles(self.request.user)
    ... # filtros opcionales por query string
    return qs.distinct()
```

Serializers diferenciados para listado y detalle

Cuando el detalle expone más campos que el listado, el ViewSet elige el serializer según la acción mediante get_serializer_class():

```

def get_serializer_class(self):
    if self.action == 'retrieve':
        return ApartmentDetailSerializer # incluye datos anidados del edificio
    return ApartmentListSerializer # vista compacta para listados

# Uso en accounts: el serializer de creacion y el de actualizacion
# exigen campos distintos, asi que tambien se selecciona por accion.
def get_serializer_class(self):
    if self.action == 'create':
        return UserCreateSerializer # incluye password (write_only)
    if self.action in ('update', 'partial_update'):
        return UserUpdateSerializer # no expone role ni status
    return UserSerializer

```

Cambios de estado auditados

El módulo de incidencias registra cada transición de estado en una bitácora de historial. Se hace sobrescribiendo `perform_update()` en el `ViewSet`:

```

# incidents/views.py - perform_update
def perform_update(self, serializer):
    estado_previo = serializer.instance.status
    instancia = serializer.save()

    # Si cambian el estado o la asignacion, deja rastro en el historial
    if (estado_previo != instancia.status) or \
        (instancia.assigned_to_id != self._asignacion_previa()):
        IncidentHistory.objects.create(
            incident=instancia,
            status_from=estado_previo,
            status_to=instancia.status,
            comment=self.request.data.get('history_comment', ''),
            changed_by=self.request.user,
        )

```

Comprobante en PDF

El comprobante de pago se genera en el servidor con ReportLab y se devuelve como respuesta HTTP en lugar de renderizar una plantilla:

```
# payments/views.py - generar_comprobante
buffer = BytesIO()
doc = SimpleDocTemplate(buffer, pagesize=letter, ...)
doc.build([...]) # elementos del PDF
buffer.seek(0)

respuesta = HttpResponse(buffer.getvalue(), content_type='application/pdf')
respuesta['Content-Disposition'] = 'inline; filename="comprobante_pago.pdf"'
return respuesta
```

Los parámetros viajan por query string (?pago_id=...&departamento=...) porque el JS externo no puede usar etiquetas de plantilla Django; los valores viajan en atributos data-* del HTML.

4. Instalación, ejecución y operación

4.1 Requisitos previos

Requisito	Detalle	Cómo verificarlo
Python	3.13 o superior (probado en 3.13.6)	python --version
pip	incluido con Python	python -m pip --version
Acceso al repositorio	git clone del remoto origin	git remote -v
Redis (opcional)	Solo si se activan Channels o Celery	redis-cli ping
PostgreSQL (opcional)	Solo para el despliegue de producción	psql --version

4.2 Instalación desde cero

Todos los comandos se ejecutan desde la raíz del proyecto, el directorio que contiene manage.py. En Windows, los comandos usan la ruta relativa del intérprete del entorno virtual.

Paso 1 - Clonar el repositorio

```
git clone https://github.com/JMJ0331/condosys.git
cd condosys
git checkout desarrollo # rama de trabajo; main es la estable
```

Paso 2 - Crear el entorno virtual

```
# Windows (PowerShell / CMD)
python -m venv venv
venv\Scripts\activate

# Linux / macOS
python3 -m venv venv
source venv/bin/activate

# Verificación
python --version # debe responder 3.13.x
```

A partir de aquí, este manual usa la forma Windows venv\Scripts\python.exe porque es el entorno verificado. En Linux/macOS se sustituye por python.

Paso 3 - Instalar dependencias

```
python -m pip install --upgrade pip
python -m pip install -r requirements.txt

# Verificación: la suite de comprobación debe reportar 0 problemas
python manage.py check
```

Paso 4 - Configurar el archivo .env

El proyecto lee su configuración sensible desde un archivo .env en la raíz, cargado por python-dotenv. El archivo está en .gitignore y nunca debe versionarse.

.env - variables disponibles

```
# .env (copiar de .env.example si existe; si no, crearlo a mano)
DJANGO_DEBUG=True
DJANGO_SECRET_KEY=<clave-larga-aleatoria>
DJANGO_ALLOWED_HOSTS=localhost,127.0.0.1

# Opcionales: solo si el despliegue los requiere
CORS_ALLOWED_ORIGINS=https://app.ejemplo.do
CSRF_TRUSTED_ORIGINS=https://app.ejemplo.do
REDIS_HOST=localhost
REDIS_PORT=6379
DATABASE_ENGINE=django.db.backends.postgresql
DATABASE_NAME=condosys_db
DATABASE_USER=postgres
DATABASE_PASSWORD=<clave>
DATABASE_HOST=localhost
DATABASE_PORT=5432
```

PENDIENTE / BUG — SECRET_KEY no tiene valor por defecto. settings.py lee SECRET_KEY = os.getenv('DJANGO_SECRET_KEY') sin fallback. Si la variable falta, Django lanza ImproperlyConfigured y el proyecto no arranca. Implica que .env es obligatorio: no se puede levantar el proyecto sin él.

Paso 5 - Aplicar migraciones

```
python manage.py makemigrations # solo si modificaste modelos
python manage.py migrate
```

Paso 6 - Cargar datos de ejemplo (opcional)

El proyecto incluye un comando de gestión idempotente que siembra registros de prueba en todas las secciones. Es la vía recomendada para levantar un entorno de demostración o para verificar permisos con los distintos roles.

Carga de datos de demostración

```
python manage.py poblar_datos

# Crea 7 usuarios base; las contraseñas de los recién creados son: Condosys123*
# admin@condosys.do rol admin
# gestion@condosys.do rol manager
# residente1..5@condosys.do rol resident
# Ejecutarlo de nuevo no duplica registros (update_or_create + transaction.atomic).
```

ATENCIÓN — No use datos de ejemplo en producción. El comando `poblar_datos` crea usuarios con contraseña conocida y datos personales ficticios. Elimínelos antes de exponer el sistema a usuarios reales.

Paso 7 - Arrancar el servidor de desarrollo

```
python manage.py runserver
# http://127.0.0.1:8000/
```

Aunque el comando se llama `runserver`, el servidor real es Daphne: la cadena `daphne` aparece primera en `INSTALLED_APPS` y Django delega en ella cuando está disponible.

4.3 Comandos de uso frecuente

Comando	Para qué sirve
manage.py runserver	Levanta el servidor ASGI de desarrollo
manage.py check	Verifica la configuración; debe dar 0 problemas
manage.py migrate	Aplica las migraciones pendientes
manage.py makemigrations	Genera migraciones tras cambiar modelos
manage.py sqlmigrate	Muestra el SQL de una migración
manage.py showmigrations	Lista el estado de las migraciones de cada app
manage.py createsuperuser	Crea el administrador inicial (pide email, no username)
manage.py shell	Consola interactiva de Python con el ORM cargado
manage.py dbshell	Consola directa de la base de datos
manage.py collectstatic	Reúne los estáticos en STATIC_ROOT para producción
manage.py test	Ejecuta la suite de pruebas (hoy vacía)
manage.py poblar_datos	Carga datos de demostración (idempotente)

Tabla 4.1 - Comandos de gestión de Django usados en el proyecto.

4.4 Flujo de trabajo con git

Concepto	Convención del proyecto
Rama de trabajo	desarrollo - aquí se desarrolla
Rama estable	main - versiones estables
Remoto	origin -> https://github.com/JMJ0331/condosys.git
Prefijos de commit	add:, fix:, update:, refactor:
Tiempo verbal	Pretérito indefinido: 'agregué', 'integré', 'arreglé'
Idioma	Español, incluidos los mensajes de commit

Ejemplo

```
fix: corregí la validación del rol en el login
git status # revisar antes de commitear
git add <archivos-específicos> # nunca commitear .env ni db.sqlite3
git commit -m "add: agregué módulo de reservas"
git push origin desarrollo
```

4.5 Rutas principales del sistema

Ruta	Módulo	Acceso
/	Inicio de sesión	Público
/salir/	Cierre de sesión (POST)	Autenticado
/inicio/	Dashboard	Todos
/mi-perfil/	Perfil del usuario	Todos
/residencial/	Ficha del residencial	Administrador
/cuentas/	Usuarios y roles	Administrador
/departamentos/	Jardines, edificios y apartamentos	Todos
/propietarios/	Propietarios	Gestión
/residentes/	Residentes y ocupantes	Gestión, Propietario
/pagos/	Pagos y cuotas	Todos (Gestión escribe)
/pagos/comprobante/	Comprobante en PDF	Todos
/mantenimientos/	Cargos de mantenimiento	Todos (Gestión escribe)
/incidencias/	Incidencias	Todos
/solicitudes/	Solicitudes y trámites	Gestión
/visitantes/	Control de visitantes	Seguridad, Gestión
/areas-comunes/	Áreas comunes	Todos
/reservas/	Reservas	Todos
/comunicados/	Comunicados	Todos
/notificaciones/	Notificaciones	Gestión
/reportes/	Reportes y bitácora	Gestión
/chat/	Mensajería	Todos
/configuracion/	Configuración (usuarios e integración IA)	Administrador
/recuperar-clave/	Solicitar recuperación de contraseña	Público
/restablecer-clave/	Restablecer contraseña con token	Público
/visitantes/buscar-cedula/	Lectura de cédula con IA (POST)	Seguridad, Gestión
/admin/	Administración de Django	Staff

Tabla 4.2 - Rutas HTML por módulo.

5. Usuarios, roles y permisos

5.1 Modelo de usuario

El sistema no usa el modelo de usuario por defecto de Django. `accounts.User` extiende `AbstractUser` con dos cambios de fondo:

Cambio	Detalle	Consecuencia
Clave primaria UUID	<code>id = UUIDField(default=uuid.uuid4)</code>	Los identificadores no son enumerables; se pueden exponer en la API
Sin username	<code>username = None</code> ; <code>USERNAME_FIELD = 'email'</code>	El login, el admin y <code>createsuperuser</code> piden email
Campo role	<code>admin</code> \ <code>manager</code> \ <code>resident</code> \ <code>propietario</code> \ <code>security</code>	El permiso se decide por rol, no por grupos
Campo status	<code>active</code> \ <code>inactive</code> \ <code>pending</code>	El login manual rechaza a quien no tenga <code>'active'</code>
Teléfono y documento	<code>phone</code> , <code>document</code> (únicos)	Identificación operativa del residente
Avatar	<code>avatar</code> (ImageField) y <code>avatar_url</code> (URLField)	Permite imagen local o remota
Vínculo a jardín	<code>garden = FK a structure.Garden (SET_NULL)</code>	Encuadre del usuario dentro del residencial

Tabla 5.1 - Diferencias del modelo de usuario frente al de Django.

5.2 Estados de la cuenta

Estado	Significado	Puede iniciar sesión
<code>active</code>	Cuenta habilitada	Sí
<code>pending</code>	En verificación, pendiente de autorización	No
<code>inactive</code>	Cuenta desactivada	No

El campo `is_active` (heredado de Django) convive con `status`. La vista de login exige las dos condiciones: `user.is_active and user.status == 'active'`. Mantenga ambas sincronizadas.

5.3 Conjuntos de roles

```
# accounts/permissions.py
ROLES_ADMIN = ('admin',)
ROLES_GESTION = ('admin', 'manager')
ROLES_SEGURIDAD = ('admin', 'manager', 'security')
ROLES_RESIDENTE = ('admin', 'manager', 'resident', 'propietario')
ROLES_TODOS = ('admin', 'manager', 'security', 'resident', 'propietario')
```

Usar siempre estas constantes en vez de escribir las tuplas a mano: si un rol se agrega después, basta con actualizar la constante.

5.4 Mecanismo de control de acceso

El sistema aplica permisos en tres capas. Todas se apoyan en el campo `role`.

Capa 1 - Decorador para vistas HTML

```
accounts/decorators.py
from accounts.decorators import role_required
from accounts.permissions import ROLES_GESTION

@login_required
@role_required(*ROLES_GESTION)
def agregar_pago(request):
    ... # renderiza el formulario
```

Si el usuario no tiene el rol, el decorador muestra un mensaje de error y redirige: a `visitantes_index` para seguridad (su único módulo) y a `inicio` en cualquier otro caso.

Capa 2 - Permisos de clase para la API

Clase	Regla	Se aplica en
IsAdmin	role in ROLES_ADMIN	UserViewSet (listar, crear), ResidencialViewSet
IsManager	role in ROLES_GESTION	Garden, Building, Propietario, Solicitud, AuditLog, Report
IsResident	role in ROLES_RESIDENTE	CommonAreaViewSet, ReservationViewSet
IsGestionOrSoloLectura	Autenticado y (método seguro o role in ROLES_GESTION)	Resident, Payment, MaintenanceCharge, AreaComun, Communication, Notification
CanModifyUser	admin, o el propio usuario	UserViewSet (detalle, editar, borrar)
CanModifyIncident	Gestión; o el que reporta; o el asignado (solo lectura)	IncidentViewSet
CanModifyVisitor	Gestión; o security; o residente del apartamento	VisitorViewSet
CanAccessApartment	Gestión; o propietario del apartamento; o residente actual	ApartmentViewSet

Tabla 5.2 - Permisos DRF personalizados definidos en `accounts/permissions.py`.

Capa 3 - Contexto de plantilla

Los context processors `modulos_usuario` y `permisos_modulo` publican en los HTML los módulos visibles y los permisos de crear, editar y eliminar de cada módulo. Es lo que permite ocultar botones según el rol sin duplicar lógica en el template.

Nombres de los context processors

```
# accounts/context_processors.py - idea central
MODULOS = [
    {'nombre': 'pagos', 'url': 'pagos_index', 'activo_en': [...]},
    {'nombre': 'incidencias', 'url': 'incidencias_index', 'activo_en': [...]},
    ... # 14 modulos en total
]
ROL_CREADOR, ROL_EDITOR, ROL_ELIMINADOR = ... # tabla de roles por modulo

# Salida al template
{
    'modulos_habilitados': [...], # que ver en el menu
    'modulos_acceso': [...],
    'puede_crear': True,
    'puede_editar': True,
    'puede_eliminar': False,
    'permisos_modulos': {...},
}
```

5.5 Matriz de permisos por rol

La tabla siguiente resume el alcance funcional de cada rol. Es la referencia para responder preguntas de soporte y para validar requisitos antes de implementar.

Capacidad	Admin	Gestión	Seguridad	Residente	Propietario
Autorizar registro de usuarios	Sí	No	No	No	No
Gestionar roles y permisos	Sí	No	No	No	No
Configurar ficha del residencial	Sí	No	No	No	No
Registrar y editar departamentos	Sí	Sí	No	No	No
Registrar propietarios	Sí	Sí	No	No	No
Registrar residentes	Sí	Sí	No	No	No
Registrar pagos y cuotas	Sí	Sí	No	No	No
Emitir comprobantes de pago	Sí	Sí	No	Sí	Sí
Consultar pagos	Sí	Sí	No	Solo los suyos	Solo los suyos
Consultar pagos pendientes y vencidos	Sí	Sí	No	No	No
Registrar cargos de mantenimiento	Sí	Sí	No	No	No
Reportar incidencias	Sí	Sí	No	Sí	Sí
Cambiar estado de incidencias	Sí	Sí	No	No	No
Ver historial de incidencias	Sí	Sí	No	Sí	Sí
Gestionar solicitudes	Sí	Sí	No	No	No
Registrar visitantes	Sí	Sí	Sí	No	No
Autorizar y controlar accesos	Sí	Sí	Sí	No	No
Administrar áreas comunes	Sí	Sí	No	No	No
Reservar áreas comunes	Sí	Sí	No	Sí	Sí
Publicar comunicados	Sí	Sí	No	No	No
Ver comunicados	Sí	Sí	Sí	Sí	Sí
Ver reportes administrativos	Sí	Sí	No	No	No
Usar la bitácora de auditoría	Sí	Sí	No	No	No
Acceder al panel /admin/	Sí	No	No	No	No

Tabla 5.3 - Alcance funcional por rol.

5.6 Matriz de acceso por módulo

Equivalente a la anterior, pero por módulo y por acción, que es como el código opera realmente.

Módulo	Ver	Crear / editar	Eliminar	Permiso DRF
Inicio	Todos	-	-	role_required
Residencial	Admin	Admin	Admin	IsAdmin
Cuentas	Admin	Admin	Admin	IsAdmin / CanModifyUser
Departamentos	Todos	Gestión	Gestión	IsManager / CanAccessApartment
Propietarios	Gestión	Gestión	Gestión	IsManager
Residentes	Gestión + propietario	Gestión	Gestión	IsGestionOrSoloLectura
Pagos	Todos	Gestión	Gestión	IsGestionOrSoloLectura
Mantenimientos	Todos	Gestión	Gestión	IsGestionOrSoloLectura
Incidencias	Todos	Todos (crear); Gestión (editar)	Gestión	CanModifyIncident
Solicitudes	Gestión	Gestión	Gestión	IsManager
Visitantes	Todos	Seguridad + Gestión	Gestión	CanModifyVisitor
Áreas comunes	Todos	Gestión	Gestión	IsGestionOrSoloLectura
Reservas	Todos	Todos	Gestión	IsResident
Comunicados	Todos	Gestión	Gestión	IsGestionOrSoloLectura
Notificaciones	Gestión	Gestión	Gestión	IsGestionOrSoloLectura
Reportes	Gestión	Gestión	-	IsManager
Chat	Todos	Todos	Todos	IsAuthenticated

Tabla 5.4 - Acceso por módulo y acción.

5.7 Sesión y autenticación

- Login por email. `settings.py` define `LOGIN_URL = '/'`, `LOGIN_REDIRECT_URL = '/inicio'` y `LOGOUT_REDIRECT_URL = '/'`.
- Redirección por rol tras entrar. Seguridad aterriza en `visitantes_index`; admin y gestor sin residencial configurado, en `residencial_index`; el resto, en `inicio`.
- Sesión en base de datos con duración de 24 horas, expiración al cerrar el navegador, cookie `HttpOnly` y `SameSite=Lax`.
- CSRF con cookie `HttpOnly` y orígenes de confianza configurables.
- API por sesión: `SessionAuthentication` es la única clase de autenticación global, y el permiso por defecto es `IsAuthenticated`.

- Validación de contraseñas: longitud mínima de 8, similitud con los datos del usuario, contraseñas comunes y sin solo dígitos.

ADVERTENCIA — El logout exige POST

`cerrar_sesion` solo ejecuta `django_logout` cuando el método es POST. Enlazarlo con un enlace `<a href>` no cierra la sesión; use un formulario con botón o JavaScript.

6. Modelo de datos

6.1 Convenciones del modelo de datos

- Clave primaria UUID en casi todos los modelos (`id = UUIDField(default=uuid.uuid4, editable=False)`). Excepción: `chat.ChatGroup`, `incidents.IncidentImage` y `reports.AuditLogDetail` usan `AutoField` implícito.
- `related_name` explícito en todas las relaciones inversas. No hay relaciones de tipo `ManyToMany` propias ni `OneToOne` en todo el proyecto.
- `on_delete` explícito en cada `ForeignKey`. Se usan los cuatro valores con significado distinto: `CASCADE`, `PROTECT`, `SET_NULL` y (en un caso) `SET_DEFAULT`.
- Índices declarados en los campos por los que se filtra o se agrupa (estado, periodo, fechas), y en los compuestos de las consultas frecuentes.
- `unique_together` en las entidades que pertenecen a un padre: un edificio no repite nombre dentro de un jardín, un apartamento no repite nombre dentro de un edificio.
- Sin lógica de negocio en `save()` salvo el caso único de `residencial.Residencial`, que documenta su excepción (ver 6.7).

6.2 Mapa relacional

Diagrama de relaciones (resumido)

```

accounts.User --SET_NULL--> structure.Garden (users)
|
+-- es referenciado por casi todos los modulos

structure.Garden --CASCADE--> structure.Building --CASCADE--> structure.Apartment
|
+-----+
| SET_NULL (owner) CASCADE PROTECT |
v           v           v           v
propietarios      residents.Resident payments.Payment   structure.Apartment
.Propietario      |           |           |
|                 | CASCADE   | CASCADE   | registered_by | SET_NULL
v                 v           \--SET_NULL--> accounts.User
accounts.User      structure.Apartment

incidents.Incident --CASCADE--> Apartment
+----CASCADE/SET_NULL--> Resident
+----PROTECT--> User (reported_by)
+----SET_NULL--> User (assigned_to)
+--> IncidentImage (CASCADE, related_name='images')
\--> IncidentHistory (CASCADE, related_name='history',
                     changed_by PROTECT sin related_name)

solicitudes.Solicitud --CASCADE--> Apartment; SET_NULL--> Resident
visitors.Visitor --CASCADE--> Apartment; PROTECT--> User (registered_by);
SET_NULL--> User (authorized_by)
maintenance.MaintenanceCharge --SET_NULL--> Apartment (NULL = aplica a todos)
communications.Communication --CASCADE--> Garden; PROTECT--> User (sender)
reservations.Reservation --CASCADE--> areas_comunes.AreaComun
SET_NULL--> Apartment / Resident
CASCADE/SET_NULL--> User (reserved_by / approved_by)
notifications.Notification --CASCADE--> User
chat.ChatMessage --CASCADE--> User (sender / receiver);
SET_NULL--> chat.ChatGroup
reports.AuditLog --CASCADE--> User
\--> AuditLogDetail (CASCADE, related_name='details')

residencial.Residencial <- modelo aislado, sin ninguna FK

```

6.3 Estructura física: Jardines, edificios y apartamentos

El núcleo espacial del sistema es una jerarquía de tres niveles que determina la organización de casi todos los demás módulos: los pagos cuelgan de un apartamento, los residentes de un apartamento, las incidencias de un apartamento.

Modelo	App	Campos principales	Relaciones
Garden	structure	name (único), location, description, is_active	building_set, users, common_areas, communications
Building	structure	name, tower, block, number_of_floors, description, is_active	garden (CASCADE, related_name='buildings')
Apartment	structure	name, floor, status, photo, is_active	building (CASCADE), owner (SET_NULL -> Propietario)

Tabla 6.1 - Jerarquía espacial.

Estados de un departamento

Valor	Etiqueta	Significado
empty	Vacío	Disponible, sinmogador asignado
occupied	Ocupado	Alquilado o en uso
maintenance	En reparación	No disponible temporalmente
blocked	Bloqueado	Inhabilitado por la administración

El estado por defecto de un apartamento nuevo es `empty`. El estado se filtra en la vista HTML mediante el parámetro `?estado=` y en la API mediante `?status=`.

6.4 Personas: propietarios y residentes

El sistema distingue dos figuras jurídicas distintas y las modela por separado:

Aspecto	propietarios.Propietario	residents.Resident
Qué representa	El dueño del inmueble	La persona que ocupa el apartamento
Clave de negocio	cedula (única)	cedula (única)
Relación con el apartamento	Apartment.owner (SET_NULL)	Resident.apartment (CASCADE, obligatorio)
Relación con el usuario	user (CASCADE, opcional)	user (CASCADE, opcional)
Datos de contacto	phone, email, emergency_contact	phone, email, emergency_contact
Estado civil	marital_status (5 opciones)	marital_status (5 opciones, duplicadas)
Datos adicionales	photo	photo, tipo_relacion, fecha_ingreso, mascotas
Orden por defecto	full_name	apartment, full_name

Tabla 6.2 - Propietario frente a residente.

Cómo leerlo: un apartamento puede tener un propietario y N residentes. El propietario no necesita estar, registrado como usuario; el residente puede ser un ocupante sin cuenta. Ambos pueden vincularse a un `accounts.User` opcional para que pueda iniciar sesión y ver su información.

Tipo de relación del residente

Valor	Etiqueta	Implicación
<code>propietario</code>	Propietario	El residente es también el dueño
<code>inquilino</code>	Inquilino	Ocupa en alquiler
<code>familiar</code>	Familiar	Pariente del titular
<code>ocupante</code>	Ocupante	Sin relación contractual (por defecto)

6.5 Dinero: cargos de mantenimiento y pagos

El módulo financiero separa el concepto de lo que se cobra del hecho de pago. Es la única forma de tener un catálogo de tarifas independiente de los movimientos reales.

	maintenance.MaintenanceCharge	payments.Payment
Qué es	El cargo o cuota a cobrar	El pago efectivamente realizado
Campos clave	concept, periodicity, amount, effective_date, payment_methods	amount, concept, period, payment_date, payment_method, status, receipt_image
Periodicidad	monthly, quarterly, semiannual, annual	No aplica (se registra por periodo)
Apartamento	Opcional. NULL significa 'aplica a todos'	Obligatorio (PROTECT)
Residente	-	Obligatorio (PROTECT)
Estado	is_active (activo/inactivo)	pending, at_risk, overdue, paid, cancelled
Orden por defecto	-effective_date	-period
Validación cruzada	-	clean(): el residente debe pertenecer al apartamento elegido

Tabla 6.3 - Catálogo de cargos frente a registro de pagos.

Conceptos de pago

Valor	Etiqueta	Tipo de obligación
<code>maintenance</code>	Mantenimiento	Cuota ordinaria
<code>extraordinary</code>	Cuota extraordinaria	Cargo adicional puntual
<code>reservation</code>	Reserva	Cargo por uso de área común
<code>parking</code>	Parqueo	Cargo fijo
<code>services</code>	Servicios	Consumo (agua, luz)
<code>other</code>	Otro	Cualquier otro concepto

Estados de pago y morosidad

Estado	Etiqueta	Significado operativo
<code>pending</code>	Pendiente	Registrado, aún no pagado, sin retraso
<code>at_risk</code>	En riesgo	Próximo a vencer; aparece en el panel de pagos por vencer
<code>overdue</code>	Vencido	Venció su periodo sin pago; entra en el reporte de morosidad
<code>paid</code>	Pagado	Cancelado
<code>cancelled</code>	Anulado	Registro invalidado, no cuenta para balances

NOTA — Cómo se modela el periodo

El campo `period` es un `DateField` que representa el PRIMER día del mes que se cobra (por ejemplo 2026-09-01 para septiembre de 2026).

Los filtros de la vista HTML (mes, año) y de la API (`month`, `year`) operan sobre ese campo; las funciones de los reportes calculan el rango de meses con esa misma convención.

6.6 Incidencias y su ciclo de vida

El módulo de incidencias es el más rico del sistema: maneja clasificación, prioridad, asignación, evidencia, notas de resolución e historial.

Ciclo de estados

Flujo de trabajo de una incidencia

[new] -> [assigned] -> [in_progress] -> [resolved] -> [closed]
Nueva Asignada En progreso Resuelta Cerrada

[rejected] (Rechazada) - salida alternativa desde new/assigned
[resolved] -> [in_progress] (reapertura)

Campo	Opciones	Uso
<code>category</code>	plumbing, electricity, structural, cleaning, security, noise, water, other	Clasificación del reporte
<code>priority</code>	low, normal, high, urgent	Prioridad de atención
<code>status</code>	new, assigned, in_progress, resolved, closed, rejected	Estado del flujo
<code>evidence</code>	FileField (upload a incidentes/evidencia/)	Archivo adjunto principal
<code>resolution_notes</code>	TextField	Descripción de la solución
<code>resolved_at</code>	DateTimeField	Momento de la resolución

Tabla 6.4 - Campos de clasificación y seguimiento de una incidencia.

Tablas Satellite

Modelo	Para qué sirve	Relación
IncidentImage	Adjuntar imágenes mediante URLs externas a la incidencia	incident (CASCADE, related_name='images')
IncidentHistory	Bitácora de cada transición de estado, con autor y comentario	incident (CASCADE, related_name='history')

ADVERTENCIA — Dos mecanismos de adjuntos coexisten

`Incident.evidence` (FileField, subida de archivo) e `IncidentImage` (tabla de URLs) cumplen la misma función. Antes de implementar, decida cuál usar y evite generar adjuntos en ambos.

6.7 Ficha del residencial

`residencial.Residencial` es un modelo conceptualmente singular: describe el residencial al que pertenece el sistema. No tiene ninguna relación con el resto de modelos.

Grupo de campos	Contenido
Identificación	nombre, rnc
Dirección	nombre_via, numero_edificacion, sector, codigo_postal, municipio, provincia
Estructura	distribucion (torres edificios), cantidad_edificios, cantidad_pisos
Contacto	telefono, correo

Grupo de campos	Contenido	Otros
		mapa_iframe (TextField para incrustar un mapa)

NOTA — La distribución es inmutable

`Residencial.save()` consulta el valor original de distribución y lo restaura si se intenta cambiar. La razón: una vez definida la estructura del residencial, cambiarla invalidaría los datos ya cargados. La UI muestra el campo como bloqueado. Consecuencia práctica: si necesita cambiar la distribución, actualice la fila directamente en la base de datos, no desde la aplicación.

6.8 Inventario de modelos por aplicación

Resumen de los 25 modelos del sistema, agrupados por aplicación.

App	Modelos	Cantidad
accounts	User (AbstractUser con PK UUID, login por email)	1
configuracion	IntegracionIA (configuración de IA, registro único)	1
login	PasswordResetToken (token de recuperación de contraseña)	1
residencial	Residencial (modelo de configuración singular)	1
structure	Garden, Building, Apartment	3
propietarios	Propietario	1
residents	Resident	1
payments	Payment	1
maintenance	MaintenanceCharge	1
incidents	Incident, IncidentImage, IncidentHistory	3
solicitudes	Solicitud	1
visitors	Visitor	1
reservations	Reservation, CommonArea (obsoleto)	2
areas_comunes	AreaComun	1
communications	Communication	1
notifications	Notification	1
reports	AuditLog, AuditLogDetail	2
chat	ChatGroup, ChatMessage	2
inicio	(sin modelos)	0

Tabla 6.5 - Modelos por aplicación (25 en total).

6.9 Decisiones de diseño del modelo

PROTECT en los vínculos con el usuario que registra

Varias relaciones usan on_delete=PROTECT para el usuario que creó el registro: incidencias (reported_by), visitantes (registered_by), comunicados (sender), pagos registrados y el historial de incidencias. La razón es de trazabilidad: no se debe poder eliminar un usuario que ha generado movimientos o cuya firma aparece en un documento.

SET_NULL donde la relación es contextual

Los vínculos que enriquecen pero no definen el registro usan SET_NULL: el propietario de un apartamento, el cargo que aplica a todos, el garden de un usuario, la asignación de una incidencia, la aprobación de una reserva.

CASCADE donde la dependencia es fuerte

Un apartamento que se elimina arrastra sus residentes, pagos, incidencias, solicitudes, visitantes y reservas. Un jardín arrastra sus edificios y, en cascada, sus apartamentos.

Restricciones de integridad a nivel de base de datos

Restricción	Modelo	Efecto
unique_together (garden, name)	Building	No se repiten nombres de edificio dentro de un jardín
unique_together (building, name)	Apartment	No se repiten nombres de apartamento dentro de un edificio
unique_together (garden, name)	reservations.CommonArea	No se repiten áreas dentro de un jardín
UniqueConstraint (common_area, start_time, end_time)	Reservation	Bloquea reservas exactamente duplicadas
UniqueConstraint (incident, url)	IncidentImage	No se repite la misma URL en una incidencia
UniqueConstraint (audit_log, key)	AuditLogDetail	No se repite una clave de detalle en una bitácora
unique=True	User.email, User.document, Propietario.cedula, Resident.cedula, Garden.name, ChatGroup.name	Claves de negocio únicas

Tabla 6.6 - Restricciones declaradas a nivel de modelo.

ADVERTENCIA — La reserva no valida solapamientos parciales

La UniqueConstraint de Reservation solo impide duplicados exactos de (common_area, start_time, end_time). Dos reservas de 10:00 a 12:00 y de 11:00 a 13:00 en la misma área pasan la validación de base de datos y colisionan. Los horarios de disponibilidad del AreaComun (available_from / available_until / available_days) tampoco se aplican a nivel de modelo: solo se validan en el código del serializer o de la vista.

6.10 Claves foráneas polimórficas manuales

Dos modelos usan un UUIDField suelto en lugar de una ForeignKey para apuntar a entidades de naturaleza variable. Esto evita una tabla puente, a costa de perder la integridad referencial:

Modelo	Campo	Apunta a	Riesgo
communications.Communication	target_id (UUID) + target_type	building_id o user_id, según target_type	Puede apuntar a un registro ya borrado
notifications.Notification	related_id (UUID)	incident_id, payment_id, etc.	Puede apuntar a un registro ya borrado

Tabla 6.7 - Referencias sin integridad referencial.

Si necesita integridad garantizada para estos vínculos, el camino es un patrón de referencia genérica (content_type + object_id) o un modelo puente por tipo.

7. API REST

7.1 Configuración global de DRF

```
# settings.py (bloque REST_FRAMEWORK que manda)
REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': [
        'rest_framework.authentication.SessionAuthentication',
    ],
    'DEFAULT_PERMISSION_CLASSES': [
        'rest_framework.permissions.IsAuthenticated',
    ],
    'DEFAULT_PAGINATION_CLASS':
        'rest_framework.pagination.PageNumberPagination',
    'PAGE_SIZE': 20,
    'DEFAULT_FILTER_BACKENDS': [
        'django_filters.rest_framework.DjangoFilterBackend',
        'rest_framework.filters.SearchFilter',
        'rest_framework.filters.OrderingFilter',
    ],
}
```

Ajuste	Valor	Implicación
Autenticación	SessionAuthentication	La API usa la cookie de sesión; no hay tokens
Permiso por defecto	IsAuthenticated	Cualquier endpoint sin permiso explícito exige sesión
Paginación	PageNumberPagination, 20 por página	Respuesta con count / next / previous / results
Filtro	DjangoFilterBackend	Campos declarados en filterset_fields
Búsqueda	SearchFilter	Campos declarados en search_fields
Orden	OrderingFilter	Campos declarados en ordering_fields

7.2 Patrón de los ViewSets

Los 21 ViewSets del sistema siguen el mismo esqueleto: queryset base filtrado por visibilidad, serializer (a veces elegido por acción), permiso de clase, y tres backends de filtro con sus campos declarados.

Anatomía de un ViewSet típico

```

class PaymentViewSet(viewsets.ModelViewSet):
    queryset = Payment.objects.all()
    serializer_class = PaymentSerializer
    permission_classes = [IsGestionOrSoloLectura]

    # Filtro por query string sobre campos del modelo
    filterset_fields = ['apartment', 'resident', 'status',
                       'payment_method', 'concept']
    # Búsqueda de texto libre
    search_fields = ['resident__full_name', 'apartment__number',
                    'resident__cedula']
    # Campos por los que se puede ordenar
    ordering_fields = ['period', 'payment_date', 'status']

    def get_queryset(self):
        # El listado se restringe a lo que le corresponde al usuario
        qs = _pagos_visibles(self.request.user)
        return qs.distinct()

```

7.3 Inventario de endpoints

Cada aplicación monta su propio DefaultRouter. No existe un prefijo global /api/: los endpoints cuelgan del prefijo de la app.

App	Prefijo	ViewSet	Acciones propias
accounts	/cuentas/	UserViewSet	login, profile, change_password, logout
residencial	/residencial/residencial/	ResidencialViewSet	-
structure	/departamentos/jardines/	GardenViewSet	-
structure	/departamentos/edificios/	BuildingViewSet	-
structure	/departamentos/departamentos/	ApartmentViewSet	-
propietarios	/propietarios/	PropietarioViewSet	-
residents	/residentes/	ResidentViewSet	-
payments	/pagos/registros/	PaymentViewSet	-
maintenance	/mantenimientos/	MaintenanceChargeViewSet	-
incidents	/incidencias/incidencias/	IncidentViewSet	-
incidents	/incidencias/historial/	IncidentHistoryViewSet (solo lectura)	-
solicitudes	/solicitudes/	SolicitudViewSet	-
visitors	/visitantes/registros/	VisitorViewSet	authorize, reject, check_in, check_out
reservations	/reservas/areas-comunes/	CommonAreaViewSet	-
reservations	/reservas/	ReservationViewSet	-
areas_comunes	/areas-comunes/api/	AreaComunViewSet	-
communications	/comunicados/	CommunicationViewSet	-
notifications	/notificaciones/	NotificationViewSet	-
chat	/chat/	ChatMessageViewSet	-
reports	/reportes/bitacoras/	AuditLogViewSet (solo lectura)	-
reports	/reportes/{resumen,pagos,ocupacion}/	ReportViewSet (acciones)	summary, payments, occupancy

Tabla 7.1 - Endpoints registrados por router.

7.4 Operaciones CRUD generadas por el router

Cada `ModelViewSet` expone automáticamente las operaciones estándar. El nombre de la URL aparece en el campo `name` de cada ruta del router:

Operación	Método HTTP	Ruta	Nombre en el router
Listar	GET	/prefijo/	<basename>-list
Crear	POST	/prefijo/	<basename>-list
Detalle	GET	/prefijo/ <pk> /	<basename>-detail
Actualizar	PUT / PATCH	/prefijo/ <pk> /	<basename>-detail
Borrar	DELETE	/prefijo/ <pk> /	<basename>-detail
Raíz de la API	GET	/prefijo/	<basename>-root (navegador)

El `<pk>` es el UUID del objeto. Las operaciones de creación, actualización y borrado solo funcionan si el permiso asignado a la acción lo permite.

7.5 Acciones personalizadas

Autenticación y perfil (accounts)

Acción	Método	Ruta	Permiso	Qué hace
login	POST	/cuentas/login/	AllowAny	Autentica por email y contraseña
profile	GET	/cuentas/profile/	IsAuthenticated	Devuelve el perfil del usuario autenticado
change_password	POST	/cuentas/change_password/	IsAuthenticated	Cambia la contraseña validando la actual
logout	POST	/cuentas/logout/	IsAuthenticated	Cierra la sesión

Ciclo de vida de un visitante (visitors)

El control de acceso de portería se implementa como acciones de detalle. Todas exigen POST y devuelven 403 si el usuario no es de seguridad o gestión.

Acción	Ruta	Estado requerido	Resultado
authorize	POST /visitantes/registros/ <pk> /authorize/	pending	Pasa a authorized y registra authorized_by
reject	POST /visitantes/registros/ <pk> /reject/	pending	Pasa a rejected
check_in	POST /visitantes/registros/ <pk> /check_in/	authorized	Registra actual_entry y pasa a completed
check_out	POST /visitantes/registros/ <pk> /check_out/	actual_entry no nula	Registra actual_exit y pasa a completed

Tabla 7.2 - Transiciones de estado de un visitante.

NOTA — Estados del visitante

Flujo normal: pending (esperando) -> authorized (autorizado) -> check_in registra la entrada real y check_out la salida. En cualquier momento antes de la entrada se puede rechazar.

Reportes (reports)

Acción	Ruta	Qué devuelve
summary	GET /reportes/resumen/	14 métricas: totales de apartamentos, ocupados, residentes activos, pagos, incidencias, visitantes, reservas e ingresos mensuales
payments	GET /reportes/pagos/	Conteo e importes por estado de pago; acepta <apartment> , <status> , <month> , <year>
occupancy	GET /reportes/ocupacion/	Conteo de apartamentos por estado (vacío, ocupado, etc.)

Tabla 7.3 - Reportes accesibles por API.

7.6 Filtros, búsqueda y ordenamiento

Los tres backends conviven en la misma respuesta. Se combinan en un único query string:

Combinando filtros en una consulta

```
GET /pagos/registros/  
?apartment=<uuid> DjangoFilterBackend (filtro exacto)  
&status=overdue DjangoFilterBackend (filtro exacto)  
&search=juan SearchFilter (búsqueda en search_fields)  
&ordering=-period OrderingFilter (campo en ordering_fields)  
&page=2 PageNumberPagination (20 por página)
```

Los `search_fields` se pueden recorrer en relaciones (`resident__full_name` significa buscar en el nombre del residente del pago). Los `ordering_fields` controlan qué campos se pueden ordenar vía query string; un campo no listado allí no es ordenable.

7.7 Visibilidad en la API

Los ViewSets no se limitan al permiso de clase: además recortan el queryset para que un usuario no pueda ver más de lo que le corresponde, aunque tenga permiso de lectura. El recorte se hace en `get_queryset()` con un filtro por usuario.

Recorte de queryset por usuario

```
def get_queryset(self):  
    user = self.request.user  
    if user.role in ('admin', 'manager'):  
        return Model.objects.all() # gestión ve todo  
    return Model.objects.filter(<lo que le toca>).distinct()
```

7.8 Formato de las respuestas

Una petición de listado devuelve la estructura de `PageNumberPagination`:

Estructura de una respuesta paginada

```
{  
    "count": 137,                // total de registros que cumplen el filtro  
    "next": "http://...page=2", // URL de la siguiente página (null si es la última)  
    "previous": null,           // URL de la anterior (null si es la primera)  
    "results": [                // la página actual de resultados  
        { "id": "...", "...": "..." },  
        ...  
    ]  
}
```

Los errores de validación devuelven el detalle por campo; los errores de permiso devuelven 403; los de autenticación, 401 (o 403 con sesión).

8. Frontend: plantillas, CSS y JavaScript

8.1 Arquitectura del frontend

El frontend es un servidor renderizado clásico, sin framework de JavaScript. Cada página se renderiza en el servidor con plantillas de Django, y el JavaScript externo solo añade interactividad (diálogos, filtros asíncronos, confirmaciones, máscaras de entrada). No hay paso de build, ni bundler, ni npm.

8.2 Plantillas

Plantilla base y componentes

Toda la interfaz se apoya en `templates/base.html`, que incluye los componentes reutilizables del directorio `templates/componentes/`:

Componente	Función
<code>base.html</code>	Esqueleto común: head, estilos, barra superior, menú lateral y bloque de contenido
<code>header.html</code>	Cabecera superior con el logotipo y el menú de usuario
<code>aside.html</code>	Menú lateral de navegación entre módulos (respeta los permisos del usuario)
<code>menu_usuario.html</code>	Desplegable del perfil con opción de cerrar sesión y editar perfil
<code>modal.html</code>	Estructura base de los diálogos (modales)
<code>botones.html</code>	Botones de acción reutilizables (guardar, cancelar, eliminar)
<code>mini_card.html</code>	Tarjeta compacta para el dashboard
<code>paginacion_tabla.html</code>	Controles de paginación reutilizables
<code>login.html</code>	Estructura de la pantalla de inicio de sesión

Tabla 8.1 - Componentes globales de plantilla.

Plantillas por módulo

Cada módulo de dominio aporta sus propias plantillas en `<app>/templates/<app>/`. La pestaña principal de cada módulo se llama siempre `index.html`; los formularios suelen estar en `agregar.html` o `nuevo.html`.

Vista	Plantilla	Contexto típico
Listado	<code>index.html</code>	<code>page_obj</code> (paginación), objeto, filtros, <code>puede_crear/editar/eliminar</code>
Formulario nuevo	<code>agregar.html</code> / <code>nuevo.html</code>	<code>form</code> (ModelForm), <code>titulo</code> , <code>accion</code>
Formulario editar	<code>agregar.html</code> / <code>nuevo.html</code>	<code>form</code> , <code>instancia</code> , <code>editing=True</code>
Perfil	<code>perfil.html</code>	<code>form</code> de datos personales

8.3 Sistema de diseño: variables.css

Los estilos se organizan en torno a un archivo de variables que es la fuente única de verdad del diseño. `static/css/variables.css` define todos los tokens (colores, sombras, bordes, radios, espaciados) y se importa desde las hojas base.

Paleta de colores

El sistema usa una paleta de verdes como color de marca, más una escala de grises para superficies y texto, y una escala de azules para elementos informativos. Los nombres siguen el patrón `--<familia>-<tono>`.

Familia	Ejemplo de variables	Uso
Verdes (marca)	<code>--green-500</code> , <code>--green-700</code> , <code>--green-900</code>	Color primario, enlaces, acciones, estados activos
Blancos (superficies)	<code>--white-50</code> a <code>--white-900</code>	Fondos de página, tarjetas, superficies elevadas
Negros (texto)	<code>--black-300</code> , <code>--black-500</code> , <code>--black-900</code>	Texto principal, texto secundario, texto deshabilitado
Azules (informativo)	<code>--blue-500</code> , <code>--blue-600</code>	Mensajes informativos, enlaces, foco

Tabla 8.2 - Familias de color del sistema de diseño.

Reglas de uso de las variables

- Todo color, sombra, borde, radio, margen y padding se escribe con una variable, nunca como valor literal repetido.
- La única unidad px permitida es para bordes y sombras (los tokens `--border-*`, `--borderR-*` y `--shadow-*` ya están en px).
- Cualquier otra medida (margen, padding, tamaño, tipografía, ancho, alto) se escribe en rem. Con `font-size: 62.5%` en `:root`, 1rem equivale a 10px.

- Los breakpoints de las media queries se mantienen en px (560px y 900px), que es la práctica estándar.
- Quien use variables debe asegurar que su hoja las importe antes (directamente o vía `style.css`).

BUENA PRACTICA — *Por qué importa*

Centralizar el diseño en variables permite cambiar la identidad visual completa del sistema editando un solo archivo, y garantiza coherencia entre módulos. Es la diferencia entre un sistema mantenible y veinte páginas que se parecen pero no encajan.

8.4 Organización de los estilos

Archivo	Ámbito	Contenido
<code>static/css/variables.css</code>	Global	Tokens de diseño (fuente única)
<code>static/css/style.css</code>	Global	Estilos base; importa <code>variables.css</code>
<code>static/css/formularios.css</code>	Global	Secciones, filas, campos, interruptores de los formularios
<code>static/css/tablas.css</code>	Global	Estilo de las tablas de datos
<code>static/css/paginacion.css</code>	Global	Controles de paginación
<code>static/css/dialogos.css</code>	Global	Modales y diálogos
<code>static/css/acciones.css</code>	Global	Paneles de acciones del dashboard
<code>static/css/entradas.css</code>	Global	Campos de entrada
<code>static/css/mediaqueries.css</code>	Global	Breakpoints comunes
<code><app>/static/css/*.css</code>	Módulo	Estilos específicos del módulo

Tabla 8.3 - Organización de las hojas de estilo.

La separación es deliberada: lo que comparten dos o más módulos vive en `static/css/` del proyecto; lo que solo usa un módulo, en su propia carpeta `static/` .

8.5 Convenciones de nombres en CSS

Toda la nomenclatura de clases está en español, con nombres claros y concisos. El proyecto mantiene un mapeo canónico desde los nombres genéricos originales a los nombres en español:

Nombre original	Nombre canónico en el proyecto
form-section	seccion-formulario
form-row	fila-formulario
form-field	campo-formulario
select-field	campo-seleccion
input-field	campo-entrada
form-switch-row	fila-interruptor
switch-label	etiqueta-interruptor
switch / switch-slider	interruptor / deslizador-interruptor
switch-input	entrada-interruptor
dialog-header	cabecera-dialogo
dialog-footer	pie-formulario
dialog-close	boton-cerrar
btn-generar-comprobante	btn-comprobante
Acciones-dashboard	panel-acciones
container-acciones	contenedor-acciones
pag-btn	boton-pagina
pag-num	numero-pagina
paginacion-numeros	numeros-paginacion

Tabla 8.4 - Mapeo de nombres de clase.

Los nombres de archivo siguen la misma línea: los estilos específicos se llaman como el módulo y en español (`pagos.css` , `departamentos.css` , `incidencias.css` , `visitantes.css`).

8.6 Reglas obligatorias del frontend

PENDIENTE / BUG — Reglas de refactor frontend (obligatorias)

1. Nada de `<script>` inline ni estilos `style=` en el HTML. El único `<script>` permitido es `<script src='...'>` para conectar el HTML con su JS. Los manejadores (`onchange`, `onclick`, ...) van dentro del archivo JS externo.
2. Estilos y JS compartidos entre apps viven en `static/css/` y `static/js/` del proyecto. Lo específico de un módulo queda en su `static/` propio.
3. El HTML principal de cada módulo se llama siempre `index.html`; los demás archivos se nombran por su contenido.
4. CSS en español, con nombres claros y concisos. Los valores salen de `variables.css`; px solo para bordes y sombras; rem para todo lo demás.
5. El JS externo no puede usar etiquetas de plantilla Django. El HTML le pasa los valores con atributos `data-*` (`data-url-comprobante`, `data-pago-apartamento`, `data-departamento`).
6. Mantener la responsividad: los estilos compartidos incluyen media queries (breakpoints 560px / 900px) usados por formularios y por la rejilla de acciones.

8.7 JavaScript

Organización

Archivo	Ámbito	Responsabilidad
<code>static/js/dialogos.js</code>	Global	Apertura, cierre y control de los modales
<code>static/js/pagos.js</code>	Global	Lógica compartida del módulo de pagos
<code>static/js/mascaras.js</code>	Global	Máscaras de entrada (teléfono, cédula, montants)
<code>static/js/contadores.js</code>	Global	Contadores animados del dashboard
<code>static/js/menu-movil.js</code>	Global	Comportamiento del menú en móvil
<code>static/js/usuario-menu.js</code>	Global	Desplegable del menú de usuario
<code>static/js/encabezado-acciones.js</code>	Global	Barra de acciones superior
<code><app>/static/js/*.js</code>	Módulo	Comportamiento específico del módulo

Cómo el JS recibe datos del HTML

El punto de fricción clásico entre plantillas Django y JavaScript son las URLs y los identificadores. La solución del proyecto es pasar los valores del servidor al script mediante atributos `data-*`, de modo que el JS nunca tenga que interpretar una etiqueta de plantilla:

Puente HTML -> JavaScript con data-*

```
<!-- HTML: el servidor pasa los valores al navegador por data-* -->
<button data-url-comprobante="{% url 'generar_comprobante' %}"
        data-pago-apartamento="{{ pago.apartment.id }}"
        data-departamento="{{ pago.apartment.name }}">
    Generar comprobante
</button>
```

```
// JS externo: lee de los data-* (sin etiquetas de plantilla)
document.querySelectorAll('[data-url-comprobante]').forEach(function (btn) {
    btn.addEventListener('click', function () {
        var url = btn.dataset.urlComprobante + '?pago_id=' + btn.dataset.pagoApartamento;
        window.open(url, '_blank');
    });
});
```

ATENCION — *Por qué esta regla existe*

Si el JS contuviera `{% url %}` o `{{ var }}`, Django lo renderizaría como texto literal dentro del archivo `.js` y el script fallaría en tiempo de ejecución. Los `data-*` evitan por completo ese problema y mantienen el JS reutilizable entre páginas.

8.8 Accesibilidad y experiencia de usuario

El proyecto aplica varias prácticas de usabilidad de forma consistente:

- Confirmación antes de acciones destructivas (eliminar) mediante diálogos.
- Mensajes de éxito y de error mediante el sistema de mensajes de Django, renderizados como notificaciones.
- Tablas ordenables y paginadas para manejar volumen de datos.
- Formularios con validación en servidor (ModelForm) y máscaras de entrada en el cliente.
- Estados vacíos y de carga considerados en los listados.
- Menú lateral que se adapta a móvil mediante JavaScript y media queries.
- Diseño responsivo con dos breakpoints principales (560px y 900px).

9. Servidor ASGI, tiempo real y tareas asíncronas

9.1 Modelo de despliegue previsto

CONDOSYS está diseñado para correr sobre una pila ASGI con Daphne como servidor principal. La entrada es `condosys/asgi.py`.

9.2 Estado actual de ASGI

El punto de entrada ASGI es deliberadamente mínimo: solo expone la aplicación Django como callable, sin enrutar websockets. Ese es el único motivo por el que los websockets no funcionan, aunque el código de channels esté escrito.

`condosys/asgi.py`

```
# condosys/asgi.py (completo)
import os
from django.core.asgi import get_asgi_application

os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'condosys.settings')

application = get_asgi_application()

# ^ Solo Django. No hay ProtocolTypeRouter.
```

Existe un módulo de routing completo en `chat/routing.py`, y los consumers están implementados en `chat/consumers.py`. Las rutas definidas son:

```
# chat/routing.py
websocket_urlpatterns = [
    re_path(r'ws/chat/(?P<user_id>[w-]+)/$',
            consumers.ChatConsumer.as_asgi()),
    re_path(r'ws/group/(?P<group_name>[w-]+)/$',
            consumers.GroupChatConsumer.as_asgi()),
    re_path(r'ws/notifications/(?P<user_id>[w-]+)/$',
            consumers.NotificationConsumer.as_asgi()),
]
```

9.3 Qué falta para activar el tiempo real

Paso	Qué hacer	Archivo
1	Añadir <code>'channels'</code> a <code>INSTALLED_APPS</code>	<code>settings.py</code>
2	Envolver la aplicación en un <code>ProtocolTypeRouter</code> con <code>'http'</code> y <code>'websocket'</code> , usando <code>AuthMiddlewareStack</code>	<code>condosys/asgi.py</code>
3	Incluir las rutas de websocket (<code>AuthMiddlewareStack(URLRouter(...))</code>)	<code>condosys/asgi.py</code>
4	Verificar que el channel layer (Redis) esté accesible	<code>settings.py</code> <code>CHANNEL_LAYERS</code>
5	Probar el handshake del websocket desde el cliente JS	cliente

Tabla 9.1 - Pasos para activar los WebSockets.

Estructura objetivo (referencia)

```
# Estructura objetivo de asgi.py
from channels.auth import AuthMiddlewareStack
from channels.routing import ProtocolTypeRouter, URLRouter
from channels.security.websocket import AllowedHostsOriginValidator
from django.core.asgi import get_asgi_application
from chat import routing as chat_routing

django_asgi_app = get_asgi_application()

application = ProtocolTypeRouter({
    'http': django_asgi_app,
    'websocket': AllowedHostsOriginValidator(
        AuthMiddlewareStack(
            URLRouter(chat_routing.websocket_urlpatterns)
        )
    ),
})
```

PENDIENTE / BUG — Punto ciego de seguridad

Los consumers existentes usan `self.scope['user']`, que solo existe si la conexión pasa por `AuthMiddlewareStack`. Sin ese middleware, el usuario no se puede resolver y el consumer fallaría al autenticar.

El `AllowedHostsOriginValidator` es igualmente necesario en producción para evitar conexiones websocket desde orígenes no autorizados.

9.4 Tareas asíncronas (Celery)

Celery está configurado en settings (broker y backend Redis, serialización JSON, zona horaria America/Bogota) pero no hay aplicación Celery implementada: no existe el archivo `celery.py` ni el inicializador de la app. Los casos de uso naturales si se quisiera activar son: envío de notificaciones por correo, avisos de pago por vencer y limpieza de sesiones.

10. Despliegue en producción

10.1 Requisitos de producción

Componente	Papel en producción
Gunicorn	Servidor WSGI de aplicación
Daphne (ASGI)	Servidor para el tiempo real (cuando se cablee)
WhiteNoise	Servidor de archivos estáticos desde el propio proceso
PostgreSQL	Base de datos de producción
Nginx / reverse proxy	Terminación TLS y proxy hacia Gunicorn
Redis	Channel layer y broker de Celery

10.2 Checklist de despliegue

#	Paso	Comando / acción
1	Rama estable	<code>git checkout main && git pull</code>
2	Entorno virtual	<code>python -m venv venv && venv\Scripts\activate</code>
3	Dependencias	<code>python -m pip install -r requirements.txt</code>
4	Archivo <code>.env</code> de producción	Configurar <code>DEBUG=False</code> , <code>SECRET_KEY</code> fuerte, <code>ALLOWED_HOSTS</code> del dominio, CORS y CSRF
5	Migraciones	<code>python manage.py migrate</code>
6	Base de datos	Cambiar <code>DATABASES</code> a PostgreSQL y descomentar el bloque
7	Estáticos	<code>python manage.py collectstatic --noinput</code>
8	Verificación	<code>python manage.py check --deploy</code>
9	Superusuario	<code>python manage.py createsuperuser</code>
10	Arranque	<code>gunicorn condosys.wsgi:application</code> (con Daphne si hay websockets)

Tabla 10.1 - Secuencia de despliegue.

ATENCIÓN — No olvide `collectstatic`

En DEBUG, Django sirve los estáticos desde las carpetas de cada app. En producción, con `DEBUG=False`, Django no los sirve: si no ejecuta `collectstatic`, la aplicación arranca pero aparece sin estilos ni JavaScript.

10.3 Conmutar a PostgreSQL

El bloque de PostgreSQL está listo y comentado en `settings.py`. Para activarlo sustituya el bloque de SQLite por el bloque de PostgreSQL y configure las variables en `.env`.

```
# settings.py - activar el bloque de PostgreSQL
DATABASES = {
    'default': {
        'ENGINE': os.getenv('DATABASE_ENGINE',
                            'django.db.backends.postgresql'),
        'NAME': os.getenv('DATABASE_NAME', 'condosys_db'),
        'USER': os.getenv('DATABASE_USER', 'postgres'),
        'PASSWORD': os.getenv('DATABASE_PASSWORD', ''),
        'HOST': os.getenv('DATABASE_HOST', 'localhost'),
        'PORT': os.getenv('DATABASE_PORT', '5432'),
        'ATOMIC_REQUESTS': True,
    }
}
```

- Instale el driver con: `python -m pip install psycopg` (ya está en requirements).
- Cree la base de datos y el usuario en PostgreSQL antes de migrar.
- `ATOMIC_REQUESTS=True` envuelve cada petición en una transacción, lo que garantiza consistencia pero puede afectar al rendimiento con operaciones pesadas.

10.4 Seguridad en producción

`settings.py` define un bloque de endurecimiento que se activa cuando `DEBUG=False`. Cada variable se lee del `.env`, de modo que puede ajustarse por entorno:

Ajuste	Propósito
<code>SECURE_SSL_REDIRECT</code>	Redirigir todo el tráfico HTTP a HTTPS
<code>SECURE_HSTS_SECONDS</code>	Duración del HSTS (31536000 = 1 año)
<code>SECURE_HSTS_INCLUDE_SUBDOMAINS</code>	Incluir subdominios en HSTS
<code>SECURE_HSTS_PRELOAD</code>	Permitir pre-carga del HSTS en el navegador
<code>SESSION_COOKIE_SECURE</code>	Cookie de sesión solo por HTTPS
<code>CSRF_COOKIE_SECURE</code>	Cookie CSRF solo por HTTPS
<code>SECURE_CONTENT_TYPE_NOSNIFF</code>	Prevenir el olfateo de tipos de contenido
<code>SECURE_REFERRER_POLICY</code>	Política de referencia (<code>same-origin</code>)
<code>CSRF_TRUSTED_ORIGINS</code>	Orígenes de confianza para formularios (dominio de producción)
<code>CORS_ALLOWED_ORIGINS</code>	Orígenes permitidos si hay consumos externos

Tabla 10.2 - Ajustes de seguridad disponibles por entorno.

PENDIENTE / BUG — *SECRET_KEY no debe quedar en el repositorio*

La clave secreta se lee del `.env` y `.env` está en `.gitignore`, lo cual es correcto. Verifique que ninguna copia de la clave se haya committeado por error antes de publicar el repositorio.

10.5 Lista de verificación de seguridad antes de publicar

#	Verificación	Estado
1	<code>DEBUG=False</code> en el <code>.env</code> de producción	[]
2	<code>SECRET_KEY</code> única, larga y no versionada	[]
3	<code>ALLOWED_HOSTS</code> limitado a los dominios reales	[]
4	<code>ALLOWED_HOSTS</code> no incluye comodines en producción	[]
5	<code>CSRF_TRUSTED_ORIGINS</code> configurado con el dominio real	[]
6	<code>SECURE_SSL_REDIRECT</code> activo (sitio bajo HTTPS)	[]
7	Cookies de sesión y CSRF marcadas como seguras	[]
8	<code>collectstatic</code> ejecutado y servido por WhiteNoise	[]
9	<code>manage.py check --deploy</code> sin avisos críticos	[]
10	Datos de ejemplo (<code>poblar_datos</code>) eliminados	[]
11	Contraseñas del superusuario cambiadas	[]
12	Redis con autenticación y no expuesto a Internet	[]

11. Problemas conocidos y deuda técnica

Este capítulo documenta los defectos y limitaciones que hoy existen en el código. Se incluye de forma deliberada: conocerlos antes de tocar el sistema evita descubrimientos en producción. Cada punto indica dónde está el problema y cuál es el camino de corrección.

11.1 Configuración (settings.py)

#	Problema	Ubicación	Corrección sugerida
1	Bloque <code>REST_FRAMEWORK</code> duplicado: el primero define throttling y el segundo lo sobrescribe. El throttling configurado nunca se aplica.	<code>settings.py</code>	Consolidar en un solo bloque, con throttling incluido
2	<code>AUTH_USER_MODEL</code> declarado dos veces	<code>settings.py</code>	Dejar una sola declaración
3	<code>STATIC_URL</code> , <code>STATICFILES_DIRS</code> y <code>STATIC_ROOT</code> definidos dos veces	<code>settings.py</code>	Eliminar el bloque duplicado
4	El bloque de seguridad <code>if not DEBUG</code> es sobrescrito por el bloque posterior que lee del <code>.env</code>	<code>settings.py</code>	Unificar en una sola lectura por entorno
5	<code>SECRET_KEY</code> sin valor por defecto; el import de <code>ImproperlyConfigured</code> no se usa	<code>settings.py</code>	Añadir fallback seguro o error explícito

Tabla 11.1 - Duplicaciones y omisiones en la configuración.

11.2 URLs

#	Problema	Efecto	Corrección sugerida
5	En 9 apps, la vista HTML <code>app_index</code> está registrada en <code>''</code> antes de <code>include(router.urls)</code> con el router en <code>r''</code> .	El listado HTML tapa el listado JSON: los endpoints <code>-list</code> y <code>-root</code> de la API quedan inalcanzables en la raíz del prefijo. El <code>-detail</code> y las acciones <code>detail=False</code> sí funcionan.	Mover el router a un prefijo propio (por ejemplo <code>api/</code>), como ya hace <code>areas_comunes</code>

Tabla 11.2 - Sombreo de rutas entre HTML y API.

Apps afectadas: `accounts`, `propietarios`, `residents`, `maintenance`, `solicitudes`, `areas_comunes` (parcialmente), `reservations`, `communications`, `notifications` y `chat`.

11.3 Tiempo real

#	Problema	Efecto	Corrección sugerida
6	<code>asgi.py</code> no usa <code>ProtocolTypeRouter</code>	Los WebSockets de chat y notificaciones no se pueden conectar	Cablear el routing (ver capítulo 9)
7	<code>channels</code> no está en <code>INSTALLED_APPS</code>	Channels no inicializa su app	Añadir la app
8	Consumers sin <code>AuthMiddlewareStack</code>	<code>self.scope['user']</code> no se resuelve	Envolver el <code>URLRouter</code> en <code>AuthMiddlewareStack</code>

Tabla 11.3 - Estado del tiempo real.

11.4 Modelo de datos

#	Problema	Ubicación	Corrección sugerida
9	<code>chat.ChatMessage.__str__</code> referencia <code>self.group_name</code> , campo que no existe: lanza <code>AttributeError</code> en todo mensaje grupal	<code>chat/models.py</code>	Usar <code>self.group.name</code> y proteger el caso group nulo
10	Dos modelos para el área común: <code>reservations.CommonArea</code> (con garden) y <code>areas_comunes.AreaComun</code> (sin garden). <code>CommonArea</code> quedó huérfano tras la migración 0003, pero sigue registrado en admin, forms, serializers, ViewSet, router y en el seed.	<code>reservations/</code> , <code>areas_comunes/</code>	Eliminar <code>CommonArea</code> y migrar sus referencias
11	<code>AreaComun</code> no tiene relación con <code>Garden</code> ni con <code>Residencial</code> : no se puede asignar un área a un jardín o edificio concreto	<code>areas_comunes/models.py</code>	Añadir la FK a <code>Garden</code>
12	PK no uniforme: <code>chat.ChatGroup</code> , <code>incidents.IncidentImage</code> y <code>reports.AuditLogDetail</code> usan <code>AutoField</code>	varios	Migrar a <code>UUIDField</code>
13	Import muerto: <code>Building</code> en <code>communications/models.py</code> ; <code>PROTECT</code> sin usar en <code>accounts</code> y <code>structure</code>	varios	Limpiar
14	<code>ChatGroup</code> no define clase <code>Meta</code>	<code>chat/models.py</code>	Añadir <code>ordering</code> y <code>verbose_name_plural</code>
15	FK polimórficas manuales sin integridad (<code>Communication.target_id</code> , <code>Notification.related_id</code>)	varios	Referencia genérica o tabla puente

Tabla 11.4 - Inconsistencias del modelo de datos.

11.5 Seguridad y permisos

#	Problema	Efecto
16	El middleware de CORS está activo pero la app <code>corsheaders</code> no está en <code>INSTALLED_APPS</code>	Las cabeceras CORS probablemente no se aplican
17	El throttling de DRF está configurado en el bloque que se sobrescribe	No hay límite de peticiones a la API
18	Permisos declarados pero sin usar: <code>IsResidentOnly</code> , <code>IsPropietario</code> , <code>IsResidentOrManager</code> , <code>IsSecurity</code> , <code>CanModifyReservation</code>	Código muerto; <code>IsResidentOrManager</code> duplica a <code>IsResident</code>
19	Permisos de objeto aplicados como permiso de clase sin <code>has_permission</code> : <code>CanAccessApartment</code> y <code>CanModifyIncident</code> no restringen los listados	El recorte real depende del filtro de <code>get_queryset</code> , no del permiso
20	Los mensajes de chat no validan rol (solo <code>login_required</code>)	Cualquier usuario autenticado puede usar el chat (consistente con la tabla de permisos)

Tabla 11.5 - Hallazgos de seguridad y permisos.

11.6 Calidad y pruebas

#	Problema	Efecto	Riesgo
21	Los <code>tests.py</code> de todas las apps son plantillas vacías; no hay suite real	Ninguna regresión se detecta automáticamente	Alto
22	No hay linter, formateador ni integración continua	El estilo depende de la disciplina individual	Medio
23	<code>requirements.txt</code> sin versiones fijadas	Instalaciones no reproducibles	Medio
24	Constantes duplicadas entre apps: <code>MARITAL_CHOICES</code> en <code>propietarios</code> y <code>residents</code> ; <code>PAYMENT_METHOD_CHOICES</code> en <code>payments</code> y <code>maintenance</code>	Cambiar una exige cambiar la otra	Bajo
25	El atributo <code>filter_fields</code> (obsoleto en DRF moderno) se usa en <code>accounts.UserViewSet</code>	Se ignora silenciosamente; el filtro por rol/estado no se aplica	Medio
26	Envío de datos de demostración: el comando <code>poblar_datos</code> crea usuarios con contraseña conocida	Riesgo si se ejecuta en producción	Alto

Tabla 11.6 - Deuda técnica y riesgos de calidad.

11.7 Orden de trabajo sugerido

Si va a abordar la deuda técnica, este orden minimiza el riesgo:

Prioridad	Tarea	Por qué primero
1	Eliminar los datos de ejemplo del flujo de despliegue y proteger el comando	Riesgo de seguridad directo
2	Arreglar el sombreado de URLs moviendo los routers a <code>/api/</code>	Devuelve la API a la normalidad sin romper la UI
3	Consolidar los bloques duplicados de <code>settings.py</code>	Reduce la confusión sobre qué configuración manda
4	Activar o retirar channels/Celery	Decidir si son capacidad real o código muerto
5	Arreglar <code>ChatMessage.__str__</code> y limpiar permisos sin uso	Corrección de bajo riesgo, alto valor
6	Escribir la suite de pruebas mínima (permisos y lógica de pagos)	Sin esto, cualquier refactor grande es arriesgado
7	Congelar versiones en <code>requirements.txt</code>	Reproducibilidad del despliegue
8	Unificar los modelos de área común	Limpieza estructural, hacerla antes de crecer

Tabla 11.7 - Backlog técnico priorizado.

12. Guía de mantenimiento y extensión

12.1 Cómo agregar un módulo nuevo

El sistema está diseñado para que agregar un módulo sea mecánico. Los pasos:

#	Paso	Detalle
1	Crear la app	<code>python manage.py startapp</code> (en español, en plural)
2	Registrar la app	Añadir <code>' '</code> a <code>INSTALLED_APPS</code> en <code>settings.py</code>
3	Definir el modelo	<code>models.py</code> con <code>UUIDField</code> como PK, <code>related_name</code> en todas las FK, índices en los campos de filtro
4	Crear migraciones	<code>python manage.py makemigrations</code> && <code>python manage.py migrate</code>
5	Definir serializers	<code>serializers.py</code> con el serializer de listado y, si difiere, el de detalle
6	Definir el ViewSet	<code>views.py</code> con el ViewSet, <code>permission_classes</code> , <code>filterset_fields</code> , <code>search_fields</code> , <code>ordering_fields</code> y <code>get_queryset</code> con el filtro de visibilidad
7	Definir permisos	Usar los permisos de <code>accounts.permissions</code> ; si hace falta uno nuevo, crearlo allí
8	Definir el contexto de permisos	Añadir el módulo a <code>MODULOS</code> y a las tablas de roles en <code>accounts/context_processors.py</code>
9	Definir rutas	<code>urls.py</code> con <code>app_index</code> (<code>@role_required</code>) y el <code>DefaultRouter</code> ; registrar en <code>ROOT_URLCONF</code>
10	Definir formularios	<code>forms.py</code> con <code>ModelForms</code> para crear y editar
11	Crear plantillas	<code>index.html</code> (listado) y <code>agregar.html</code> / <code>nuevo.html</code> (formulario), extendiendo <code>base.html</code>
12	Crear estilos	<code><app>/static/css/<app>.css</code> importando <code>variables.css</code> ; reutiliza las clases globales
13	Crear scripts	<code><app>/static/js/<app>.js</code> ; sin script inline, sin estilos inline
14	Registrar en el admin	<code>admin.py</code> con los <code>ModelAdmin</code> correspondientes
15	Probar	<code>python manage.py check</code> , luego recorrer el flujo con cada rol

Tabla 12.1 - Checklist para agregar un módulo.

12.2 Cómo agregar un campo a un modelo

```
# 1. Modificar el modelo
class Incident(models.Model):
    ...
    categoria_seguridad = models.CharField(max_length=50, blank=True)

# 2. Crear y aplicar la migración
python manage.py makemigrations incidents
python manage.py migrate

# 3. Actualizar el serializer si el campo debe exponerse en la API
class IncidentSerializer(serializers.ModelSerializer):
    class Meta:
        model = Incident
        fields = '__all__' # o la lista explícita

# 4. Si el campo debe filtrarse en la API, declararlo
filterset_fields = [..., 'categoria_seguridad']
```

ADVERTENCIA — Cuidado con los `on_delete` al agregar relaciones

Al agregar una ForeignKey, especifique siempre `on_delete` de forma explícita. `CASCADE` borra en cascada, `PROTECT` impide el borrado y `SET_NULL` deja el campo vacío. Elegir por defecto puede tener consecuencias irreversibles sobre los datos.

12.3 Buenas prácticas al escribir código en este proyecto

- Idioma: todo el código, comentarios, nombres de plantilla y mensajes van en español, salvo los nombres de clase y método de Django/DRF.
- Modelos: `UUIDField` como PK, `related_name` explícito, `on_delete` explícito, índices en los campos de filtro, `unique_together` en lo que pertenece a un padre.
- Permisos: usar las constantes de cuentas (`ROLES_*`) y los permisos de `accounts.permissions.py`; nunca escribir la tupla de roles a mano.
- Visibilidad: todo listado debe pasar por un helper `_<entidad>visibles(user)` y terminar en `.distinct()` cuando hay joins.
- Serializers: un serializer de listado ligero y uno de detalle cuando el detalle enriquece; seleccionarlos con `get_serializer_class` según la acción.
- Frontend: sin script inline, sin estilos inline, CSS en español, tokens de `variables.css`, `rem` excepto bordes y sombras.
- JS y plantillas: pasar datos por `data-*`, nunca por etiquetas de plantilla dentro del JS.
- Paginación: 15 elementos por página en la vista HTML; 20 en la API (el default de DRF).

12.4 Cómo diagnosticar problemas

Síntoma	Causa probable	Qué revisar
No arranca: <code>ImproperlyConfigured</code>	Falta <code>DJANGO_SECRET_KEY</code> en <code>.env</code>	El archivo <code>.env</code> y que <code>python-dotenv</code> lo esté leyendo
No arranca: No module named X	Dependencia no instalada o app no registrada	<code>pip install -r requirements.txt</code> ; <code>INSTALLED_APPS</code>
Login correcto pero redirige mal	<code>status</code> distinto de <code>'active'</code>	El campo <code>status</code> del usuario
La API devuelve HTML en vez de JSON	El listado HTML sombrea el listado JSON	El orden de las rutas en <code>urls.py</code> (ver 11.2)
Los estilos no cargan en producción	<code>collectstatic</code> no ejecutado	<code>STATIC_ROOT</code> y WhiteNoise
Un filtro de la API no hace nada	El campo no está en <code>filterset_fields</code> , o se usó <code>filter_fields</code> (obsoleto)	Declarar el campo en <code>filterset_fields</code>
El JS no encuentra un valor	El HTML no pasa el <code>data-*</code> esperado	Revisar que el JS lea <code>dataset</code> y que el HTML tenga el atributo
Un color no coincide con el diseño	Se usó un literal en vez de una variable	Revisar el uso de <code>variables.css</code> y las reglas de px/rem
Los websockets no conectan	<code>asgi.py</code> no enruta websockets	Cablear el <code>ProtocolTypeRouter</code> (capítulo 9)
<code>check</code> reporta 0 pero algo falla	El bloque de configuración que manda no es el que se lee	Revisar duplicados en <code>settings.py</code>

Tabla 12.2 - Diagnóstico rápido por síntoma.

12.5 Comandos de verificación antes de publicar

```
# 1. Verificar la configuración (debe reportar 0 problemas)
```

```
python manage.py check
```

```
# 2. Revisar la configuración de producción
```

```
python manage.py check --deploy
```

```
# 3. Verificar que no hay cambios de modelo pendientes
```

```
python manage.py makemigrations --check --dry-run
```

```
# 4. Ver el estado de las migraciones
```

```
python manage.py showmigrations
```

```
# 5. Ejecutar la suite de pruebas
```

```
python manage.py test
```

12.6 Resumen de la arquitectura en una línea

CONDOSYS es un monolito modular de Django: un único proyecto y una única base de datos, dieciocho aplicaciones de dominio que se comunican exclusivamente a través del modelo de usuario compartido y de las relaciones entre modelos, una capa de presentación renderizada en el servidor con un sistema de diseño basado en variables, y una API REST en paralelo que expone cada módulo con los mismos datos y los mismos permisos por rol. Toda la lógica de negocio vive en el lado del servidor; el JavaScript solo aporta interactividad. El sistema está preparado para escalar hacia tiempo real y base de datos de producción, pero ambas rutas requieren trabajo adicional documentado en los capítulos 9 y 10.

13. Módulos y funciones incorporados en 2026

Este capítulo documenta las piezas que se añadieron al sistema después de la primera versión del manual y que no aparecían en los capítulos anteriores: la aplicación `configuracion`, sus modelos, la recuperación de contraseña por correo y la lectura de cédulas con inteligencia artificial. Debe leerse como complemento de los capítulos 3, 5, 6 y 7.

13.1 La aplicación `configuracion`

`configuracion` es una app nueva que centraliza la administración de cuentas de usuario y la conexión del sistema con un proveedor de inteligencia artificial. A diferencia del resto de apps, **no expone API REST**: no tiene `ModelViewSet` ni `DefaultRouter`; toda su capa de presentación son vistas de función protegidas.

Elemento	Valor
Ruta base	<code>/configuracion/</code> (registrada en <code>condosys/urls.py</code> , antes de <code>login.urls</code>)
Presentación	Templates <code>configuracion/index.html</code> , <code>crear.html</code> y <code>editar.html</code>
Estáticos	<code>configuracion/static/configuracion/</code> (CSS y JS propios)
Acceso	Exclusivo del rol <code>admin</code> (todas las vistas usan <code>@role_required('admin')</code>)
API REST	No. Solo vistas HTML de función

Vistas y rutas (`configuracion/urls.py`):

Nombre de URL	Ruta	Vista	Función
<code>configuracion_index</code>	<code>/configuracion/</code>	<code>app_index</code>	Página principal con pestañas y listado de usuarios
<code>crear_usuario</code>	<code>/configuracion/agregar/</code>	<code>crear_usuario</code>	Alta de usuario con contraseña temporal
<code>actualizar_usuario</code>	<code>/configuracion/actualizar/<uuid:pk>/</code>	<code>actualizar_usuario</code>	Edición de un usuario
<code>eliminar_usuario</code>	<code>/configuracion/eliminar/<uuid:pk>/</code>	<code>eliminar_usuario</code>	Baja de usuario (con diálogo de confirmación)
<code>guardar_ia</code>	<code>/configuracion/guardar-ia/</code>	<code>guardar_ia</code>	Guarda proveedor, modelo y API Key de la integración IA

Tabla 13.1 - Rutas y vistas de la app configuración.

La pantalla `configuracion_index` se organiza en seis pestañas. Solo **Usuarios** e **Integración AI** están implementadas; **Seguridad**, **Notificaciones**, **Comprobantes fiscales** y **Preferencias generales** muestran el texto "Módulo próximamente". La opción **Configuración** del menú de usuario (no del menú lateral) apunta a `configuracion_index` y solo se renderiza para el rol `admin` (`templates/componentes/menu_usuario.html`). El módulo **Residencial** del menú lateral sigue existiendo de forma independiente.

Protecciones de la vista de edición (`actualizar_usuario`): al editar la propia cuenta se fuerza `role='admin'` , `status='active'` e `is_active=True` para no bloquear el acceso, y `eliminar_usuario` impide borrar la cuenta con la que se está trabajando.

Formularios (`configuracion/forms.py`):

Formulario	Campos	Reglas
UsuarioForm	first_name, last_name, document, avatar, phone, email, role, status, is_active	email y document únicos (validación propia)
UsuarioCreateForm	Los anteriores + password y password_confirmation	Contraseña mínima de 8 caracteres; se hashea con <code>set_password</code>
IntegracionIAForm	proveedor, modelo, api_key	<code>api_key</code> opcional; en blanco conserva la guardada

Tabla 13.2 - Formularios de la app configuracion.

13.2 Modelo `configuracion.IntegracionIA`

Registro único (singleton lógico) que guarda la conexión con un proveedor de IA. Tiene PK `UUIDField`.

Campo	Tipo	Descripción
<code>proveedor</code>	CharField (choices)	<code>gpt</code> (GPT · OpenAI), <code>claude</code> (Claude · Anthropic), <code>gemini</code> (Gemini · Google), <code>llama</code> (Llama · Meta), <code>mistral</code> (Mistral AI), <code>deepseek</code> (DeepSeek), <code>grok</code> (Grok · xAI). Por defecto <code>gpt</code>
<code>modelo</code>	CharField	Identificador del modelo (texto libre con sugerencias)
<code>api_key</code>	CharField	Clave de la API; puede quedar en blanco
<code>MODELOS_POR_PROVEEDOR</code>	dict de clase	Modelos con visión por proveedor (ver tabla siguiente)

Tabla 13.3 - Modelo IntegracionIA.

Proveedor	Modelos con visión
gpt	gpt-5, gpt-4o, gpt-4.1, gpt-4o-mini, gpt-4.1-mini
claude	claude-opus-4, claude-sonnet-4, claude-3-7-sonnet, claude-3.5-sonnet
gemini	gemini-2.5-pro, gemini-2.5-flash, gemini-2.0-flash, gemini-1.5-pro
mistral	pixtral-large, pixtral-12b, mistral-medium-3, mistral-small-3
grok	grok-4, grok-3
deepseek, llama	Sin modelos con visión

Tabla 13.3.1 - Modelos con visión por proveedor.

Métodos relevantes: `obtener_unico()` devuelve el registro único de configuración (o `None`) y se usa tanto en la vista como en `ia_vision`. El campo `api_key` es un `PasswordInput` con

`render_value=True` y botón de ojo para mostrar u ocultar la clave.

13.3 Modelo `login.PasswordResetToken`

Token de un solo uso que respalda la recuperación de contraseña. PK `UUIDField`.

Campo	Tipo	Descripción
<code>user</code>	FK a <code>User</code> (CASCADE)	Cuenta que solicita el restablecimiento (<code>related_name='password_reset_tokens'</code>)
<code>token</code>	CharField(64), único, indexado	Valor generado con <code>secrets.token_hex(32)</code>
<code>created_at</code>	DateTimeField	Fecha de emisión
<code>expires_at</code>	DateTimeField	Vencimiento (emisión + 30 minutos)
<code>used_at</code>	DateTimeField (nullable)	Marca de uso; <code>NULL</code> mientras no se haya consumido

Tabla 13.4 - Campos de `PasswordResetToken`.

Meta ordena por `-created_at` e incluye el índice compuesto (`user`, `used_at`). Los métodos `esta_usado`, `expirado()`, `es_valido()` y `marcar_como_usado()` concentran la lógica de vigencia.

13.4 Recuperación de contraseña por correo

El flujo se compone de dos vistas de función en `login/views.py`.

Nombre de URL	Ruta	Vista	Responsabilidad
<code>recuperar_clave</code>	<code>/recuperar-clave/</code>	<code>solicitar_recuperacion</code>	Solicita el correo y envía el enlace
<code>restablecer_clave</code>	<code>/restablecer-clave/</code>	<code>restablecer_clave</code>	Valida el token y guarda la nueva contraseña

Tabla 13.5 - Rutas del flujo de recuperación.

Secuencia:

1. El usuario abre `/recuperar-clave/` desde el enlace "¿Se te ha olvidado la clave?" del login y escribe su correo.
2. `solicitar_recuperacion` busca `User` con `is_active=True` y `status='active'`. Si existe, elimina los tokens activos previos, crea uno nuevo (`secrets.token_hex(32)`) que vence en 30 minutos (`DURACION_TOKEN_MINUTOS = 30`) y envía el correo con el enlace a `/restablecer-clave/?token=...`.
3. La respuesta es siempre el mismo mensaje genérico, exista o no la cuenta, para evitar la enumeración de usuarios.

4. `restablecer_clave` comprueba que el token no esté usado ni vencido, valida que las dos claves coincidan y pasen `validate_password`, guarda con `set_password` y llama a `marcar_como_usado()`.

Envío del correo. `login/correos.py::enviar_correo_recuperacion` usa la biblioteca **Resend**. Requiere las variables de entorno `RESEND_API_KEY` y `RESEND_FROM` (valor por defecto `CONDOSYS<br><onboarding@resend.dev>`). Si falta la API key lanza `RuntimeError`; en modo `DEBUG` registra el enlace en el log en lugar de fallar silenciosamente. El asunto es "Solicitud de restablecimiento de contraseña".

NOTA — Relación con `settings.py`

El bloque "Email Configuration" (`EMAIL_BACKEND`, `EMAIL_HOST`, etc.) sigue presente para correo SMTP genérico; el envío transaccional de recuperación usa específicamente Resend mediante `RESEND_API_KEY` / `RESEND_FROM`.

13.5 Lectura de cédula con inteligencia artificial

El módulo de Visitantes incorpora la búsqueda por cédula a partir de una foto. La lógica vive en `configuracion/ia_vision.py` y se invoca desde `visitors/views.py::buscar_cedula`.

Elemento	Valor
Ruta	<code>/visitantes/buscar-cedula/</code> (nombre <code>buscar_cedula</code> , POST multipart)
Imagen máxima	8 MB (<code>MAX_IMAGEN_CEDULA_MB</code>)
Procesamiento	<code>configuracion/ia_vision.py</code> con Pillow: valida la imagen y la reduce a JPEG (lado máximo 1280 px)
Prompt	Pide los 11 dígitos de la cédula dominicana (formato 000-0000000-0) o <code>NADA</code>
Timeout	60 segundos
Resultado	Redirige a <code>/visitantes/?documento=<cedula></code> para filtrar el listado

Tabla 13.6 - Lectura de cédula con IA.

`extraer_digitos()` toma de la respuesta del modelo los primeros 11 dígitos, o `None` si no hay cédula legible. La función depende de que la app `configuracion` tenga una `IntegracionIA` con `api_key`; si no, `ia_vision` lanza `ErrorVision`. En la plantilla, el diálogo "Buscar por cédula" permite usar la cámara del dispositivo o adjuntar un archivo.

13.6 Variables de entorno nuevas

Variable	Uso	Valor por defecto
RESEND_API_KEY	Clave privada de Resend	Vacío
RESEND_FROM	Remitente verificado del correo	CONDOSYS <onboarding@resend.dev>

Tabla 13.7 - Variables de entorno del correo transaccional.

13.7 Impacto en los inventarios

- La app `configuracion` se suma a las aplicaciones del proyecto y aporta el modelo `IntegracionIA`.
- La app `login` deja de estar vacía: aporta `PasswordResetToken`.
- El total de modelos pasa de 23 a **25** (ver la tabla del capítulo 6.8, actualizada).
- La tabla de rutas principales (capítulo 4.5) incorpora `/configuracion/`, `/recuperar-clave/`, `/restablecer-clave/` y `/visitantes/buscar-cedula/`.
- La matriz de permisos (capítulo 5.5) incluye el módulo **Configuración** como acceso exclusivo del rol `admin`.